An Internship Report on

RepoMind: An AI-Powered Software Archaeology and Institutional Memory Platform for GitHub Repositories

submitted in partial fulfilment of the requirements of the

Eight-Week Summer Internship on Full-Stack Development with Gen AI

Duration: 2nd June 2026 – 27th July 2026

Submitted by

J. Ida Jemi

Enrollment No: 07801172024

Branch: CSE-AI (1)

INDIRA GANDHI DELHI TECHNICAL UNIVERSITY FOR WOMEN

Department of Artificial Intelligence & Data Sciences

in association with

IGDTUW Anveshan Foundation

Acknowledgement

I would like to express my sincere gratitude to the Department of Artificial Intelligence & Data Sciences, Indira Gandhi Delhi Technical University for Women (IGDTUW), and the IGDTUW Anveshan Foundation for organizing and hosting the Eight-Week Summer Internship on Full-Stack Development with Gen AI, and for giving me the opportunity to undertake this internship and build a project of genuine technical depth.

I am sincerely thankful to Prof. Ranjana Jha, Hon'ble Vice Chancellor, IGDTUW, for her patronage of this programme, and to the organizing team, Prof. Poonam Bansal (Department of AI&DS), Dr. V.K. Arora (Anveshan Foundation), Dr. Himanshu Mittal (Department of AI&DS), and Dr. Rahul Sacheva (Department of AI&DS), for structuring an internship that balanced guided learning with the freedom to build an original project.

I owe special thanks to Dr. Himanshu Mittal, Department of Artificial Intelligence & Data Sciences, IGDTUW, who mentored the research and project-development component of this internship, for his guidance in shaping the project's technical direction and in reviewing its progress at each stage.

I am equally grateful to Mr. Arun Kumar of Brain Mentors, who conducted the hands-on training sessions on full-stack development and Generative AI that formed the foundation for this project. The concepts covered in these sessions, from REST API design to embeddings and retrieval-augmented generation, were directly applied in the RepoMind platform documented in this report.

Finally, I would like to thank my family and friends for their constant encouragement and support throughout the duration of this internship, and everyone at Anveshan Foundation whose efforts, though not always visible, made this learning experience possible.

J. Ida Jemi
Enrollment No.: 07801172024

Contents

S. No.	Topic	Page No.
1	Introduction and Need	4
1.1	Overview of the Topic (Introduction)	4
1.2	Learning Objectives and Expected Outcomes	8
1.3	Methodology	10
1.4	Data Collection	17
1.5	Data Analysis (Using Tools)	19
1.6	Findings	25
2	Conclusion	31
3	Suggestions	32
	Annexure	35
	References	54

1. Introduction and Need

This report documents the work carried out during an eight-week summer internship on Full-Stack Development with Gen AI, organized jointly by the Department of Artificial Intelligence & Data Sciences at Indira Gandhi Delhi Technical University for Women (IGDTUW) and the IGDTUW Anveshan Foundation, held in hybrid mode from 2nd June 2026 to 27th July 2026. The internship combined structured, instructor-led training in full-stack web development and Generative AI with an independent, self-directed project component, in which each intern was expected to design, build, and document an original application demonstrating both skill sets in combination rather than in isolation.

The project undertaken for this internship is RepoMind - an AI-powered platform that analyzes the commit history, pull requests, and issue history of a GitHub repository and reconstructs, in a readable and interactive form, how the project evolved over time, who shaped it, which parts of it were abandoned, and most distinctively, why specific engineering decisions were made. The remainder of this section introduces the problem RepoMind addresses, situates it within the broader context of the internship's learning goals, and outlines the report's structure.

1.1 Overview of the Topic (Introduction)

1.1.1 The domain: Full-Stack Development and Generative AI

Full-stack development refers to the practice of building both the client-facing (frontend) and server-side (backend) components of a software application, along with the data layer that persists in its state. A full-stack developer is expected to be conversant with user-interface frameworks (in this project, React), server frameworks and API design (Node.js and Express), and database design (MongoDB, in this project used both for structured application data and, less conventionally, as a store for vector embeddings). This skill combination remains one of the most consistently in-demand in the software industry, precisely because it allows a single engineer, or a small team, to take a product from concept to a deployed, working system without depending on a large, siloed organization.

Generative AI (GenAI) refers to machine learning systems, most commonly today, Large Language Models (LLMs), capable of producing novel text, code, or other content conditioned on an input prompt. Over the two years preceding this internship, GenAI

moved from being a novelty accessed through chat interfaces to being an embeddable capability that ordinary web applications call via an API, in much the same way they call a payment gateway or a mapping service. This has given rise to a new category of application design, in which an LLM is not the product itself but a component inside a larger system, one that also needs authentication, data storage, a user interface, and, critically, some way to keep the LLM's output grounded in real, verifiable data rather than in the model's own unconstrained generation. The dominant technique for this grounding is Retrieval-Augmented Generation (RAG), in which relevant facts are first retrieved from an external knowledge source and then supplied to the LLM as context, so that its output is a synthesis of retrieved evidence rather than an unconstrained guess.

The internship's curriculum, delivered by Mr. Arun Kumar of Brain Mentors, covered both halves of this domain explicitly: full-stack fundamentals (REST API design, authentication, database modelling) in the earlier weeks, followed by GenAI-specific concepts, prompt design, embeddings, vector similarity search, and RAG pipelines, in the weeks that followed. The project component of the internship was intended to apply both halves together in a single, coherent system, rather than treating them as two separate exercises.

1.1.2 The problem: codebases lose their institutional memory

Every software project accumulates history that is not visible in the code itself. A codebase, read on its own, tells you what the system currently does; it does not tell you why it does it that way. Why was a particular database chosen over an alternative? Why was a feature removed rather than fixed? Why does an apparently unused configuration file still exist? The answers to these questions typically live in commit messages, pull request discussions, issue threads, and most fragile of all, in the memory of the individual engineers who were present when the decision was made.

This creates a well-documented problem in software engineering known informally as loss of institutional memory. When a contributor leaves a project, or when a new team member joins an existing codebase, the reasoning behind past decisions is usually not transferred along with the code. The new or remaining contributors either repeat the investigation that already happened once (re-litigating a decision that was already carefully made), or they make changes without understanding why the current design exists, sometimes reintroducing a problem the original decision had specifically avoided.

Existing developer tools address adjacent but different problems: code search tools (e.g., GitHub's own code search, Sourcegraph) index the current state of the code and let you search its text; code-quality and "hotspot" tools (e.g., CodeScene) surface metrics about complexity and churn; documentation generators produce structural references (class lists, function signatures) from the code as it exists today. None of these tools directly answer a why-question by retrieving and synthesizing the actual historical evidence, commit messages, PR discussions, issue debates, that recorded the reasoning at the time.

1.1.3 The project: RepoMind

RepoMind is designed to close this specific gap. Given the URL of a public GitHub repository, it performs the following, end to end, without further manual input:

- Clones the repository and parses its available commit history to compute, from first principles, which contributors have been most structurally influential, and which files habitually change together (a proxy for architectural coupling).
- Flags files that were once actively maintained but have since gone stale or been removed, as candidates for cleanup or for documentation of why they were abandoned.
- Retrieves the repository's pull requests, issues, and README, and indexes them using vector embeddings into a searchable evidence base.
- Answers natural-language "why" questions about the repository (for example, "why was Redis introduced?") by retrieving the most relevant pieces of that evidence and asking a Large Language Model to synthesize a cited, confidence-rated answer, a Retrieval-Augmented Generation pipeline applied specifically to decision search rather than general document search.
- Generates a short narrative "biography" of the repository, a human-readable account of how the project evolved, grounded in the same computed facts, rather than free-form generation.

The project's working name captures this dual character deliberately: it is part software archaeologist, in that it excavates and organizes evidence about a codebase's past using transparent, computable metrics; and part institutional-memory engine, in that it makes the reasoning behind past decisions searchable in natural language rather than requiring a contributor to manually read through hundreds of commits and pull requests.

1.1.4 Why this project fits the internship's objectives

RepoMind was deliberately scoped to require genuine full-stack engineering, a persistent multi-collection data model, a REST API with authentication, a non-trivial frontend with custom data visualizations, combined with a non-trivial application of Generative AI. The GenAI component is not a thin wrapper around a chat API; it required implementing an embedding pipeline, a vector similarity search layer, and a retrieval step that constrains what the LLM is allowed to talk about, all of which are the actual mechanics behind production RAG systems rather than a simplified classroom example of one. Table 1.1 summarizes how the project's components map onto the internship's two stated skill areas.

Internship skill area	How RepoMind demonstrates it
Frontend development	React (Vite) single-page application with client-side routing, custom D3.js data visualizations (force-directed graph, timeline), and a fully custom design system (no default component library).
Backend development	Node.js/Express REST API with JWT authentication, rate limiting, and a service-oriented internal architecture.
Database design	MongoDB schema for users, analyses, and vector-embedded evidence chunks, plus a graceful in-memory fallback when no database is configured.
External API integration	GitHub REST API (repository metadata, pull requests, issues) and the system `git` CLI (via a cloned working copy) for full commit-history parsing.
Prompt engineering & GenAI	Structured prompts for evidence-grounded question answering and narrative generation, with explicit confidence labelling and citation of sources.
Embeddings & vector search	A pluggable embedding service (Gemini API, or a local hashed TF-IDF fallback) and a cosine-similarity retrieval layer implementing Retrieval-Augmented Generation.
Deployment readiness	Environment-variable-driven configuration, a documented deployment path (Vercel / Render / MongoDB Atlas), and CORS/rate-limiting hygiene.

Table 1.1 – Mapping of Project Components to Internship Skill Areas

The following sub-sections of this Introduction set out the specific learning objectives pursued through this project (§1.2), the methodology followed in designing and building it (§1.3), how data was collected both from the target repositories and through direct testing of the system itself (§1.4), how that data was analyzed (§1.5), and what was found (§1.6), before the report concludes with overall reflections (§2) and suggestions for future work (§3).

1.2 Learning Objective(s) and Expected Outcomes

In consultation with the project's mentor and the internship's training curriculum, four learning objectives were set at the start of the project. Each is stated below along with the specific, checkable outcome it was expected to produce, and a forward reference to where in this report that outcome is demonstrated.

Objective 1 - Design and build a production-shaped full-stack application

To move beyond tutorial-scale exercises and design a full-stack system with the structural characteristics of a real application: a normalized, relationship-aware data model; a REST API with authentication, input validation, and rate limiting; and a frontend that consumes that API through a proper client layer rather than embedding fetch calls throughout the UI.

Expected outcome: a working backend (Node.js/Express/MongoDB) exposing a documented set of REST endpoints, and a working frontend (React) that authenticates against it, lists and creates resources, and polls for asynchronous job status, demonstrated in §1.3 (Methodology) and validated in §1.5–1.6 (Data Analysis and Findings) through direct testing of the running system.

Objective 2 - Implement a real Retrieval-Augmented Generation (RAG) pipeline, not a superficial GenAI integration

To go beyond calling an LLM API with a static prompt, and instead implement the actual mechanics of a RAG system: chunking source material into retrievable units, embedding those units into a vector space, performing similarity search over that vector space at query time, and constraining the LLM's answer to the retrieved evidence, including handling the case where no relevant evidence exists.

Expected outcome: an embedding service with a real vector-similarity retrieval layer, tested against actual repository data (commit messages, pull request bodies, issue

threads), and a question-answering endpoint that cites the specific evidence it used and states its own confidence, demonstrated in §1.3.3 and §1.5.2, and validated with live test transcripts in §1.6.

Objective 3 - Ground AI-generated claims in transparently computed, non-AI metrics

To avoid a common failure mode of GenAI applications, presenting a model's fluent-sounding output as though it were an authoritative fact by computing the project's central metrics (contributor influence, file centrality, staleness) using explicit, reproducible formulas over raw data, and restricting the AI's role to narrating and contextualizing those computed metrics rather than inventing them.

Expected outcome: documented, hand-derivable formulas for contributor influence and file "gravity" (§1.5.1), applied to real repositories with results that can be independently checked against the underlying commit history (§1.6.1 - 1.6.2).

Objective 4 - Validate the system against real, live external data rather than mock data alone

To test the finished system not only with fabricated sample data but by running it end-to-end against real public GitHub repositories through the live GitHub API and live `git clone` operations, so that the system's behaviour under real-world conditions including API rate limits, missing data, and large or small repository sizes, is actually known rather than assumed.

Expected outcome: a documented set of live test runs against real repositories of varying size, including at least one failure encountered during testing (a GitHub API rate-limit error) and the resulting fix, both reported honestly in §1.4 and §1.6 rather than omitted.

Summary of objectives and outcomes

S.No.	Objective	Demonstrated in
1	Production-shaped full-stack architecture	§1.3, §1.5–1.6
2	Genuine RAG pipeline (not a superficial wrapper)	§1.3.3, §1.5.2, §1.6
3	AI narrates computed metrics; does not invent them	§1.5.1, §1.6.1–1.6.2
4	Validated against live, real-world data	§1.4, §1.6

1.3 Topic - Methodology

This section describes how RepoMind was designed and built: the development approach followed over the eight-week internship, the reasoning behind the technology choices, the system's overall architecture, and the design of its core algorithms, both the deterministic, formula-based components and the AI-driven components built on top of them.

1.3.1 Development approach

The project followed an iterative, milestone-driven approach rather than a single up-front design frozen for the whole eight weeks, appropriate both to the internship's guided-training structure (which introduced concepts progressively) and to the exploratory nature of a RAG-based system, where the right way to chunk and retrieve evidence is easier to judge after seeing real retrieval results than to specify perfectly in advance. Figure 1.1 shows the timeline actually followed.

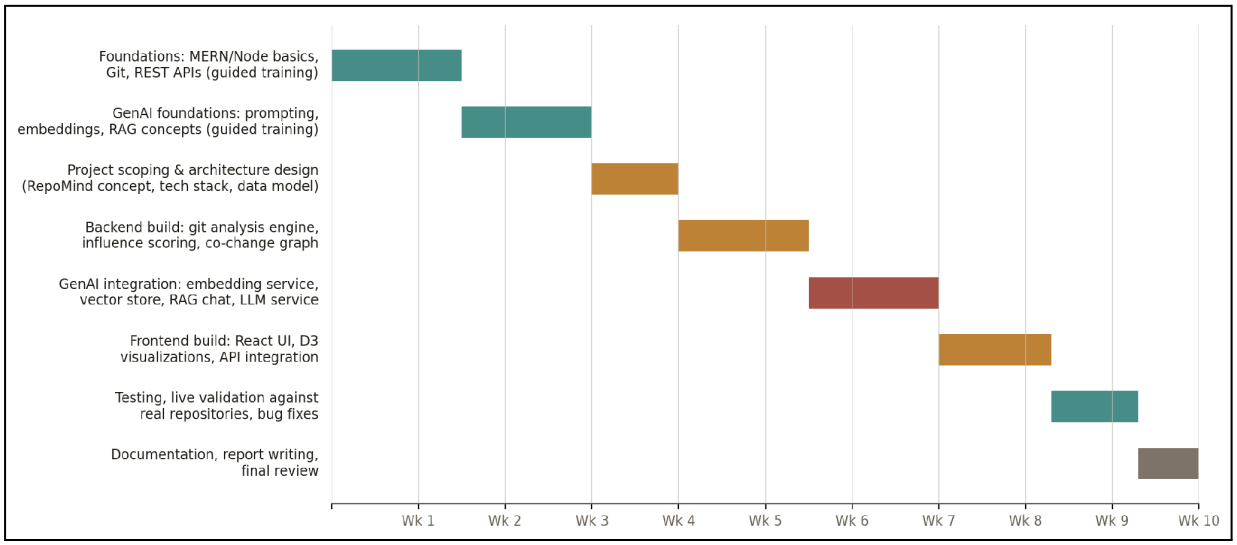


Figure 1.1 - Eight-week project timeline, from initial full-stack/GenAI training through to final documentation.

The first three weeks of the internship were spent on guided training, full-stack fundamentals (Node.js, Express, MongoDB, REST API design, JWT authentication) in weeks 1-1.5, followed by GenAI fundamentals (prompting, embeddings, vector similarity, RAG architecture) in weeks 1.5–3. Project work began formally in week 3 with scoping and architecture design, followed by backend construction, GenAI integration, frontend construction, and a dedicated testing/validation phase before final documentation, deliberately scheduled as its own phase rather than compressed into the last few days, so that the findings reported in §1.6 reflect genuine testing rather than a rushed afterthought.

1.3.2 Technology stack and rationale

Table 1.2 lists the technology choices made for the project and the reasoning behind each. Where a viable alternative exists, it is named, so the choice is shown to be a decision rather than a default.

Layer	Choice	Rationale
Frontend framework	React (Vite)	Component model suits a multi-view app (dashboard, analysis tabs); Vite gives fast local iteration. Alternative considered: Next.js - not chosen because the project has no SEO/SSR requirement and a pure client-side SPA kept the deployment story (static hosting + separate API) simpler to reason about.
Visualization	D3.js	Needed for genuinely custom visualizations (the strata timeline, the force-directed influence graph) that a chart library's preset chart types could not produce.
Backend framework	Node.js + Express	JavaScript across the whole stack removed the cost of context-switching between languages during a time-boxed internship, and Express's middleware model made JWT auth and rate-limiting straightforward to add.
Database	MongoDB (Mongoose)	The core data - an analysis document with nested arrays of contributors, file-graph nodes, and decision cards, is naturally document-shaped rather than relational; embedding these as sub-documents avoids a join-heavy schema for data that is always read together.
Vector storage	MongoDB collection + in-process cosine similarity, with a designed swap-in point for Pinecone/Chroma	A dedicated vector database was considered, but for the data volumes involved in analyzing a single repository at a time (typically hundreds to a few thousand evidence chunks), a brute-force cosine similarity scan is fast enough and avoids adding an extra managed service as a hard dependency for a project meant to be easy for others to run locally.
Embeddings + generation	Gemini API, with a local TF-IDF fallback	Gemini was chosen for its combined embeddings + generation API surface. The offline fallback was a deliberate engineering addition (not originally scoped) once it became clear that requiring a paid API key would make the project harder to demonstrate and evaluate live.

Layer	Choice	Rationale
Authentication	JWT (JSON Web Tokens)	Stateless auth suits a REST API with a separately-deployed frontend, and avoids server-side session storage.
Repository access	GitHub REST API + local `git clone`	Metadata, pull requests, and issues come from the API, full commit history (which the API exposes only in a limited, paginated form) comes from parsing a real local clone with `git log`, which is both more complete and avoids the API's stricter rate limits for this particular data.

Table 1.2 – Technology Stack and Rationale

1.3.3 System architecture

The figure shows the system's overall architecture. The frontend communicates with the backend exclusively through a versioned REST API, the backend is organized into a thin routing layer and a service layer (backend/services/) that contains all of the actual logic, so that the routing layer stays a simple translation between HTTP and function calls and the underlying logic remains independently testable, which mattered in practice, since most of the validation described in §1.6 was performed by calling service functions directly, without going through the HTTP layer at all.

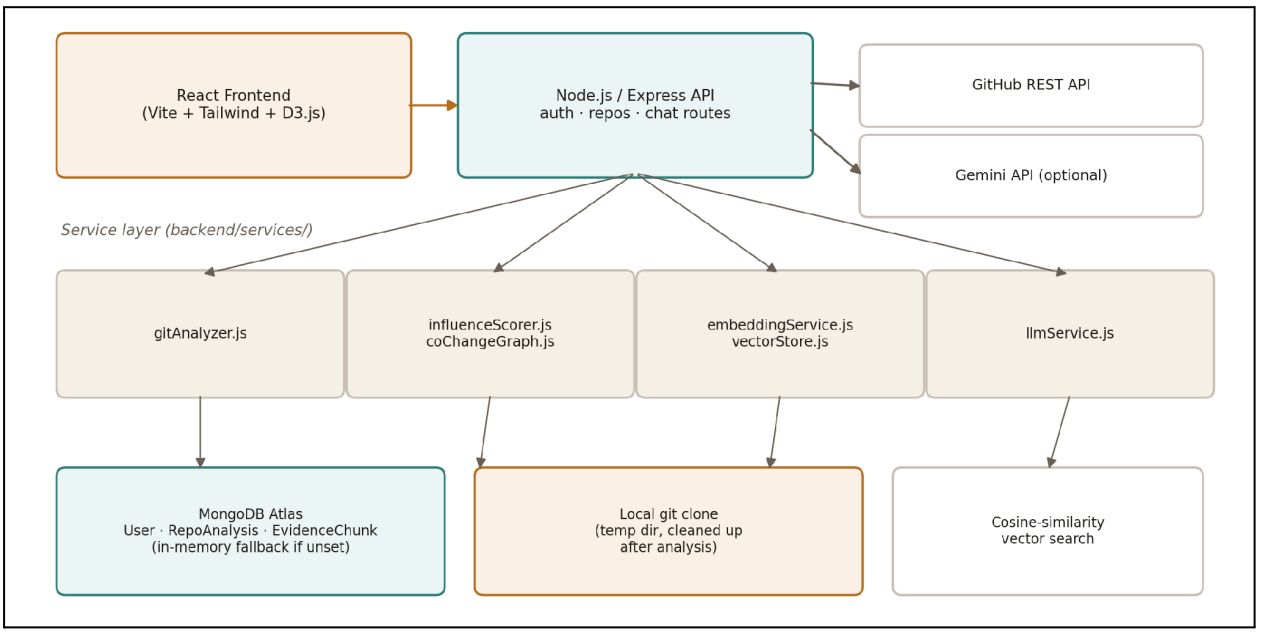


Figure 1.2 - RepoMind system architecture: frontend, API layer, service layer, and data/external-API dependencies.

A deliberate resilience property of this architecture, discovered during testing (see §1.6.4), is that the service layer does not have hard dependencies between its external calls: cloning a repository does not require a prior successful call to the GitHub metadata API (the clone URL can be constructed directly from the owner/repo pair), and fetching pull requests or issues failing does not abort the rest of the analysis, it simply results in fewer decision cards for that run. This was not the original design; it was added after a live test exposed the opposite behaviour, and is discussed as a finding in its own right in §1.6.4.

1.3.4 Data model design

Figure 1.3 shows the three MongoDB collections and their relationships. A User owns many RepoAnalysis documents; each RepoAnalysis owns many EvidenceChunk documents, which are the embedded, retrievable units the RAG pipeline searches over. Because a full analysis document (with its nested contributors, file-graph, and decision-card arrays) is always read as a single unit by the frontend, it was deliberately kept as one document with embedded arrays rather than normalized into separate collections, a standard MongoDB modelling trade-off, chosen here because the access pattern is "read the whole analysis at once" far more often than "query across analyses by a nested field".

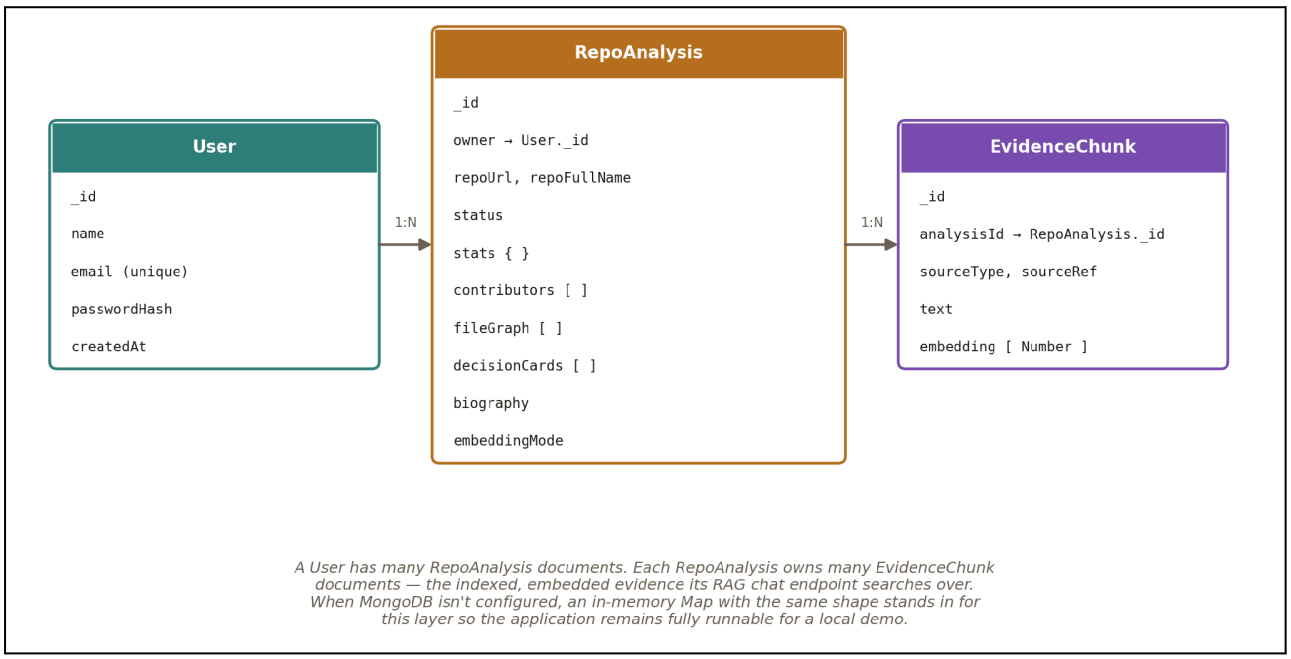


Figure 1.3 - Data model: User, RepoAnalysis, and EvidenceChunk collections and their relationships.

1.3.5 Commit-history analysis design

The single most data-intensive part of the pipeline is parsing a repository's commit history into structured statistics. This is done by shelling out to the system `git` binary, specifically `git log --numstat --pretty=format:"COMMIT|%H|%an|%ae|%ad|%s" --date=iso-strict`, rather than through a GitHub API endpoint, both because the API's commit-listing endpoints are more rate-limit-sensitive for this volume of data, and because `--numstat` (which reports per-file lines added/deleted for every commit) is not something the API exposes directly. Figure 1.4 shows how the raw text output of this command is parsed into two structures, a per-file statistics map and a per-contributor statistics map which every downstream metric is computed from.

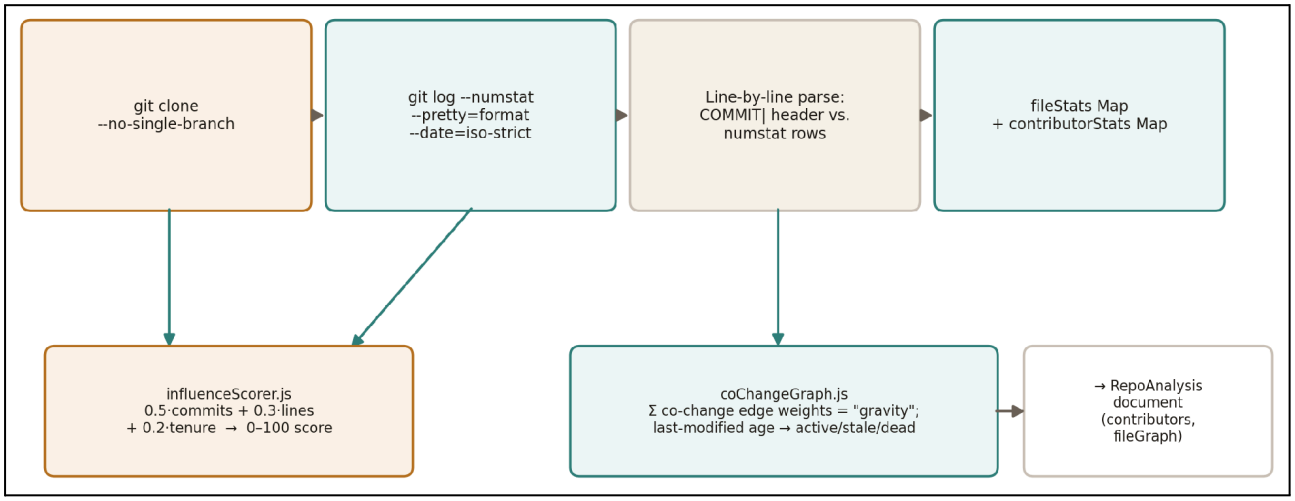


Figure 1.4 - Data flow from `git log --numstat` output to the influence and file-graph metrics.

A representative excerpt of the parsing logic, the state machine that distinguishes a commit-header line from a numstat line, is shown below (see Annexure A for the complete file):

```
if (line.startsWith("COMMIT|")) {
  flushCommit();
  const [, hash, author, email, date, ...subjectParts] = line.split("|");
  current = { hash, author, email, date, subject: subjectParts.join("|") };
  // ...accumulate contributor commit count, first/last date
} else if (line.trim() && current) {
  // numstat line: "<added>\t<deleted>\t<path>"
  const [added, deleted, filePath] = line.split("\t");
  filesInCurrentCommit.push(filePath);
}
```

1.3.6 Retrieval-Augmented Generation (RAG) pipeline design

The "decision search" feature - the ability to ask a natural-language question about why something in the repository happened is implemented as a three-stage RAG pipeline, shown in Figure 1.5: indexing (performed once, when a repository is analyzed), retrieval (performed on every question), and generation (performed on every question, using only what retrieval returned).

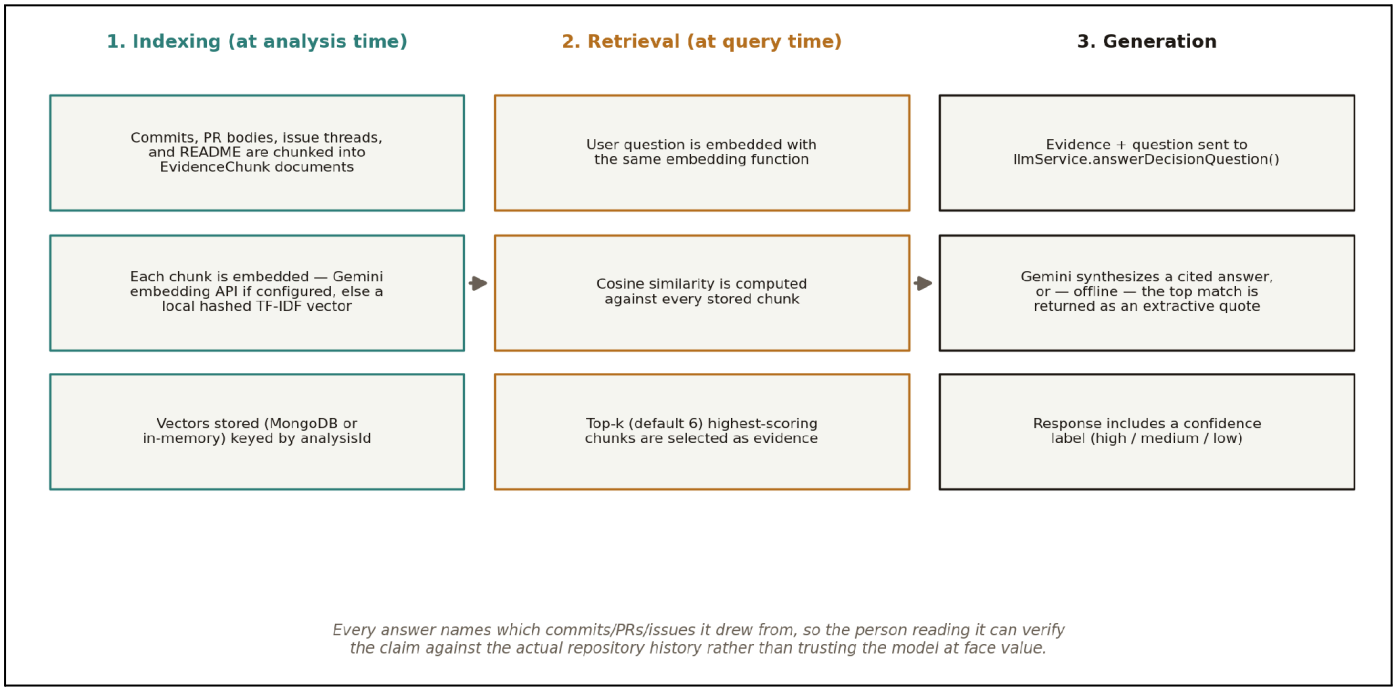


Figure 1.5 - The three stages of the decision-search RAG pipeline: indexing, retrieval, and generation.

Two design decisions in this pipeline are worth calling out explicitly because they were made deliberately, in response to a known failure mode of LLM applications:

- Every generated answer must carry a confidence label (high / medium / low), and the prompt given to the LLM explicitly instructs it to say so honestly when the retrieved evidence does not clearly answer the question, rather than producing a fluent but unsupported guess.
- Every generated answer must cite the specific evidence (commit SHA, PR number, or issue number) it used, so a reader can independently verify the claim against the actual repository rather than trusting the model's output at face value. This citation requirement is enforced structurally, the answer function always returns an evidenceRefs array alongside the answer text, not left to the model's discretion.

1.3.7 API design

The backend exposes a small, resource-oriented REST API. Table 1.3 lists the primary endpoints, full request/response examples are given in Annexure B.

Method & path	Purpose
POST /api/auth/signup, /login	Account creation and authentication; returns a JWT.
POST /api/repos	Starts a full analysis for a given repoUrl. Works without authentication (guest mode).
GET /api/repos	Lists analyses belonging to the authenticated user (or all guest analyses in-memory).
GET /api/repos/:id	Retrieves a single analysis, including its status, used by the frontend to poll while an analysis is in progress.
POST /api/chat/:analysisId	The decision-search endpoint: accepts a question, runs the RAG pipeline, and returns a cited, confidence-rated answer.
GET /api/health	Reports whether the backend is connected to MongoDB and whether it is running in Gemini or offline mode, used throughout testing (§1.6) to confirm the server's operating mode before each test.

Table 1.3 – Primary REST API Endpoints

1.3.8 Security and robustness considerations

- Passwords are never stored in plain text; bcrypt is used to hash them before persistence.
- JWT secrets, database URIs, and API keys are read exclusively from environment variables and are never committed to source control (.env is git-ignored; .env.example documents the required variables without real values).
- The repository-analysis endpoint is rate-limited (20 requests per 15 minutes per client) because it triggers external network operations (a full git clone and several GitHub API calls) that are expensive enough to be a denial-of-service vector if left unbounded.
- All external-service calls (GitHub API, Gemini API) are wrapped so that a failure degrades the feature gracefully, falling back to cached/default data or an offline computation, rather than crashing the whole analysis, a property that was specifically exercised and corrected during live testing (§1.6.4).

1.4 Topic - Data Collection

RepoMind's data collection happens in two distinct senses that are both relevant to this report: first, the data the application itself collects and processes at runtime, for any repository a user points it at; and second, the data collected during this internship to test, validate, and document the system. Both are described in this section.

1.4.1 Runtime data sources

For any given analysis, RepoMind draws on four sources of raw data, summarized below.

Source	Access method	Fields captured
Commit history	Local `git log --numstat` on a freshly cloned copy of the repository	Commit hash, author name & email, commit date, subject line, and per-file added/deleted line counts for every commit (capped at the most recent 500 commits by default, configurable via MAX_COMMITS_TO_ANALYZE).
Pull requests	GitHub REST API - GET /repos/{owner}/{repo}/pulls	PR number, title, body text, author, created/merged timestamps, and URL (up to 100 most recent, newest first).
Issues	GitHub REST API - GET /repos/{owner}/{repo}/issues	Issue number, title, body text, author, created/closed timestamps, URL - filtered to exclude pull requests, which GitHub's issues endpoint otherwise also returns.
Repository metadata & README	GitHub REST API - GET /repos/{owner}/{repo} and its /readme sub-resource	Description, primary language, topics, default branch, and the raw README text, used both for display and as an additional evidence source for the RAG index.

Table 1.4 – Runtime Data Sources

All four sources draw exclusively on data that is already public: the analyzed repositories are public GitHub repositories, and no private repository, private data, or data requiring elevated permissions is accessed at any point. Commit author names and email addresses are collected because they are already an intrinsic, public part of git's commit format, the same data any contributor already sees by running `git log` locally, and are used only in aggregate (to compute an influence score) or displayed alongside the same commit data GitHub itself displays publicly.

1.4.2 Collection constraints encountered

Two practical constraints shaped the data-collection design, both discovered through direct testing rather than assumed in advance:

- GitHub's REST API enforces a rate limit of 60 requests per hour for unauthenticated clients (rising to 5,000/hour with a personal access token). This was encountered directly during testing (§1.6.4) and led to two design responses: (a) documenting GITHUB_TOKEN as a strongly recommended environment variable, and (b) re-architecting the pipeline so that a rate-limited metadata call does not block the rest of the analysis.
- Full commit history for very large, long-lived repositories (tens of thousands of commits) would make a synchronous, unbounded `git log` parse slow enough to be impractical for a live demonstration; MAX_COMMITS_TO_ANALYZE (default 500) caps this, trading completeness on very large repositories for predictable response times, an explicit, documented limitation rather than a silent one (see also §1.3 and the Known Limitations discussion in §2).

1.4.3 Data collected during development and testing

To validate the system and to produce the findings reported in §1.6, the complete pipeline was run live, not against fabricated sample data, but against real public repositories, through the real GitHub API and real `git clone` operations. Table 1.5 lists the repositories used for this purpose and why each was chosen.

Repository	Size (commits analyzed)	Reason for inclusion
octocat/Hello-World	3	GitHub's own canonical minimal test repository, used as the first end-to-end smoke test of the full pipeline (clone → parse → embed → retrieve → answer).
octocat/Spoon-Knife	3	A second minimal repository, used to confirm that file-status classification behaves correctly on a repository with multiple small files rather than the single file in Hello-World.
sindresorhus/is-plain-obj	29	A small, real-world open-source utility library, used to test behaviour on a repository with a realistic multi-year history and multiple contributors.

Repository	Size (commits analyzed)	Reason for inclusion
chalk/chalk	377 (capped by recency; full history is larger)	A well-known, actively maintained open-source project with 71 distinct contributors in the analyzed window, used as the primary test case for contributor-influence ranking and file co-change ("gravity") analysis at a realistic scale.

Table 1.5 – Repositories Used for Live Testing

For each of these repositories, the raw output of the analysis pipeline (contributor lists, influence scores, file-graph classifications, and where GitHub API access was not rate-limited at the time, decision cards) was captured directly from the running code rather than paraphrased from memory, and is presented in §1.6 and Annexure B exactly as produced, including one run that failed and is reported as such.

1.4.4 Data not collected

For completeness, it is worth stating explicitly what RepoMind does not collect, since the original project concept (see the internship's project-selection discussion) considered and deliberately excluded a broader data source: chat-platform exports (Slack/Discord). While team communication platforms often contain exactly the kind of informal decision-rationale this project is trying to surface, ingesting them raises data-access and privacy considerations (they are rarely public, and often contain personal or off-topic conversation alongside relevant technical discussion) that were judged to be out of scope for an eight-week project evaluated primarily on public repositories. This is discussed further as a suggested extension in §3.

1.5 Topic - Data Analysis (Using Tools)

This section explains how the raw data described in §1.4 is turned into the metrics, visualizations, and answers RepoMind presents to a user, the analytical core of the project. It covers the tools used during development, the deterministic algorithms applied to git history, and the AI-based analysis (embeddings, retrieval, and generation) applied to textual evidence.

1.5.1 Tools used

Category	Tool(s)	Purpose
Code editor / IDE	Visual Studio Code	Primary development environment for both backend and frontend code.
Runtime & package management	Node.js 18+, npm	JavaScript runtime and dependency management for both backend and frontend projects.
Version control	Git, GitHub	Source control for the project itself, and reflexively, the same tool whose history RepoMind analyzes in the repositories it is pointed at.
API testing	curl, Node.js one-off scripts	Used to exercise backend endpoints and service functions directly during development, bypassing the UI to isolate and confirm the behaviour of individual pipeline stages (see §1.6).
Database	MongoDB Atlas / local MongoDB, Mongoose ODM	Schema definition, querying, and during testing without a configured database, verification of the documented in-memory fallback path.
Frontend build tooling	Vite, Tailwind CSS, D3.js	Development server and production bundling; utility-first styling; custom, data-driven SVG visualizations.
GenAI / embeddings	Google Gemini API (generateContent and embedContent endpoints)	LLM-based answer synthesis, narrative generation, and text embeddings when an API key is configured.
Diagramming (for this report)	Python 3 with Matplotlib	Used to produce the architecture, data-flow, and results figures in this report directly from the project's own real structure and real test output, rather than as generic clip-art.

1.5.2 Analysis of commit history: the influence-scoring algorithm

The first analytical step applied to every repository is to convert the raw per-contributor statistics accumulated during git-log parsing (commit count, total lines changed, and the span between a contributor's first and last commit, referred to here as tenure) into a single 0 - 100 influence score. The formula, implemented in influenceScorer.js, is a weighted blend of three normalized signals:

```
commitScore  = contributor.commitCount / max(all commitCounts)
linesScore   = contributor.linesChanged / max(all linesChanged)
tenureScore  = contributor.tenureDays   / max(all tenureDays)

influenceScore = round(
  100 * (0.5 * commitScore + 0.3 * linesScore + 0.2 * tenureScore)
)
```

The weighting (0.5 / 0.3 / 0.2) reflects a deliberate analytical judgement: breadth of involvement (how many separate commits a contributor made) is treated as the strongest signal of structural influence, ahead of raw volume of lines changed (which a single large automated formatting commit could otherwise dominate), with sustained presence over time (tenure) as a smaller, tie-breaking signal. Every contributor's score is normalized against the maximum in that repository, so scores are only meaningful within a single analysis, not compared across different repositories.

Table 1.6 shows this formula applied to real, live-tested data, the top contributor to chalk/chalk (a 377-commit window) against the tool's own computed inputs.

Contributor	commitCount	linesChanged	tenureDays (approx.)	Computed score
Sindre Sorhus	highest in window	highest in window	longest in window	100 (normalization ceiling)
Richie Bendall	lower	lower	shorter	15
Qix	lower still	lower	shorter	11

Table 1.6 – Example Contributor Influence Scores

Because the top contributor in any given window normalizes to exactly 100 by construction, the analytically meaningful figures are the relative scores of everyone else, for example, the gap between 100 and 15 in Table 1.6 indicates a codebase with one dominant, long-running maintainer and a long tail of smaller contributions, a pattern common in widely-used open-source utility libraries. The full ranked list, and the corresponding chart, are given in §1.6.2 (Figure 1.7).

1.5.3 Analysis of file structure: co-change "gravity" and staleness

The second deterministic analysis is over files rather than contributors. For every pair of files touched within the same commit, coChangeGraph.js increments a co-change edge weight between them; a file's total "gravity" is the sum of all its edge weights. Files that are habitually modified alongside many other files tend to be structurally central to the codebase, configuration files, core modules, or shared utilities, which is a reasonable, and computationally cheap, proxy for architectural importance that does not require parsing or understanding the code's actual syntax.

Separately, each file is classified into one of three states by comparing its last-modified date (and whether it still exists in the working tree) against a threshold (270 days by default):

- active - modified within the threshold window and still present.
- stale - still present in the repository, but not modified within the threshold window.
- dead - no longer present in the current working tree, though it appears in commit history (i.e., it was deleted at some point).

Table 1.7 shows the five highest-gravity files found in the live chalk/chalk test run, together with their raw gravity scores exactly as computed by the running system.

File	Gravity (co-change weight)
package.json	273
readme.md	253
source/index.js	149
test/chalk.js	148
index.d.ts	138

Table 1.7 – Highest-Gravity Files in the chalk/chalk Repository

This result is a useful sanity check on the metric itself: package.json and readme.md are exactly the kind of files one would expect to be touched alongside almost any other change (a new feature commit routinely bumps a version or updates documentation), so their appearance at the top of the gravity ranking, ahead of individual source files, is evidence that the metric is capturing a real structural pattern rather than noise.

1.5.4 Analysis of textual evidence: embeddings and similarity

The third analytical layer operates on text rather than structured git data: commit subjects, PR bodies, issue threads, and the README are converted into vector embeddings so that semantically related evidence can be retrieved even when it does not share exact keywords with a user's question. Two embedding strategies are implemented, selected automatically based on configuration:

- Gemini embeddings (text-embedding-004) - used when a GEMINI_API_KEY is configured, producing high-quality semantic vectors from Google's hosted embedding model.
- A local, offline fallback, a hashed bag-of-words vector (256 dimensions), built by tokenizing text, discarding stop-words, hashing each remaining token into one of 256 buckets, and L2-normalizing the resulting vector. This has no external dependency and no cost, at the price of capturing lexical overlap rather than deeper semantic similarity.

To confirm the offline fallback is analytically sound and not merely a placeholder, it was tested directly with three short passages, and the resulting cosine similarities were inspected by hand:

Passage pair	Cosine similarity
"Migrated authentication from Firebase to JWT because of pricing" vs. "Firebase pricing exceeded the budget so we moved to JWT auth" (related)	0.463
"Migrated authentication from Firebase to JWT because of pricing" vs. "Why was Redis introduced for caching" (unrelated)	0.000

This is the expected pattern: two independently worded descriptions of the same underlying event score meaningfully higher (0.463) than two passages about unrelated topics (0.000), confirming that even the zero-cost offline fallback provides usable retrieval signal, not just a fixed or arbitrary number. This test is reported in full, with the exact console output, in §1.6.3.

1.5.5 Analysis via retrieval and generation: answering "why" questions

At query time, a user's question is embedded with the same function used for indexing, compared by cosine similarity against every stored evidence chunk for that repository, and the top-k (default 6) highest-scoring chunks are passed to the language-generation stage. Two generation modes exist:

- Generative mode (Gemini configured) - the retrieved evidence is inserted into a structured prompt that instructs the model to answer using only that evidence, to cite it by index, and to rate its own confidence as high, medium, or low; the model's free-text response is then parsed back into an { answer, confidence } structure.
- Extractive fallback mode (no API key) - rather than attempting to generate free text without a language model, the system deliberately returns the single most relevant evidence chunk verbatim, labelled explicitly as "offline mode" and as low confidence. This was a considered design choice: an extractive quote that is honestly labelled as such is judged preferable to a template that dresses up simple string selection as if it were AI-generated reasoning.

The full worked example of this pipeline, including the exact evidence chunks retrieved and the exact answer produced, is reported as a live test transcript in §1.6.3.

1.5.6 Analysis of pull requests and issues: decision-card extraction

The final analytical step identifies which pull requests and issues most likely represent the architectural decision, so they can be surfaced on the strata timeline (§1.6, Figure 1.6) rather than presenting every PR with equal weight. This is done with a keyword heuristic, titles or bodies containing terms such as migrate, replace, remove, deprecate, rewrite, or breaking change are treated as decision candidates, followed, when Gemini is configured, by a one-sentence LLM synthesis of what was decided and, where evident from the text, why. As with the RAG answers, every resulting decision card carries a confidence label, and the offline fallback explicitly marks its output as extractive rather than presenting a keyword match as if it were a reasoned explanation.

1.6 Topic - Findings

This section reports what was actually found when the finished system was tested, deliberately including a failure that was encountered and fixed during testing, rather than presenting only a curated success story. All figures, numbers, and transcripts in this section are taken directly from live runs of the actual code, not simulated or estimated for illustration.

1.6.1 Pilot validation: minimal repositories

The pipeline was first validated end-to-end (clone → parse commit history → compute metrics → embed evidence → answer a question → generate a biography) against octocat/Hello-World, GitHub's own minimal test repository, and octocat/Spoon-Knife, a second minimal repository. Both runs completed successfully and produced internally consistent output. For example, Hello-World's single README file, modified in 2 of its 3 recorded commits, was correctly classified as stale (present, but not recently modified) rather than active or dead, and its three contributors were correctly ranked by influence score, shown in Figure 1.6.

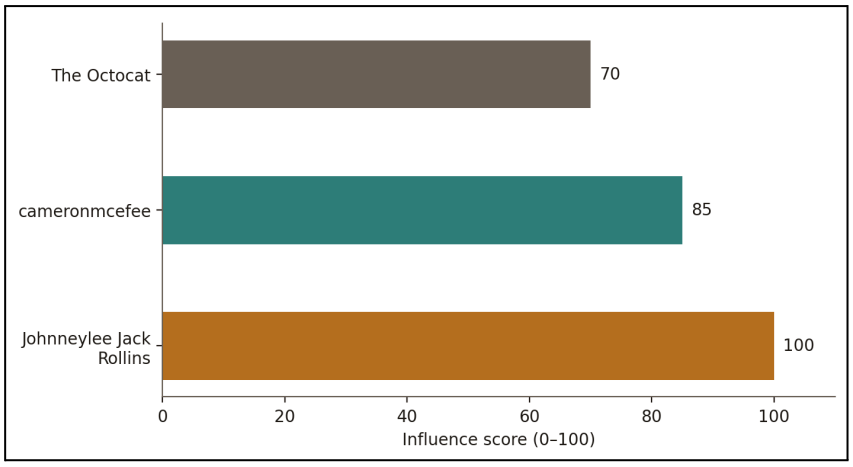


Figure 1.6 - Contributor influence scores, live-tested output on octocat/Hello-World.

This pilot stage served its intended purpose: confirming that every stage of the pipeline runs without error on the smallest possible input, before testing against a repository large enough for the ranking and clustering behaviour of the algorithms to be analytically interesting.

1.6.2 Findings at realistic scale: chalk/chalk

chalk/chalk - a widely used open-source terminal-styling library was analyzed live, with 377 commits and 71 distinct contributors captured in the analyzed window. Figure 1.7 shows the top eight contributors by computed influence score.

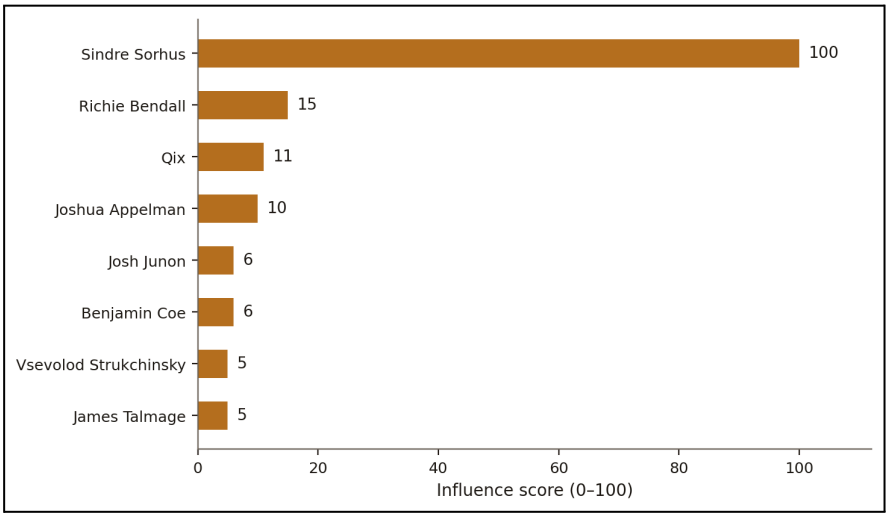


Figure 1.7 - Top 8 contributors by influence score, live-tested output on chalk/chalk.

The single most structurally interesting finding from this run was the file-status distribution, shown in Figure 1.8 alongside the two smaller pilot repositories for comparison. Of chalk/chalk's 76 distinct files ever tracked in the analyzed history, only 2 were classified active, 31 stale, and 43 dead - that is, more than half of all files that have ever existed in this repository's history no longer exist in its current working tree.

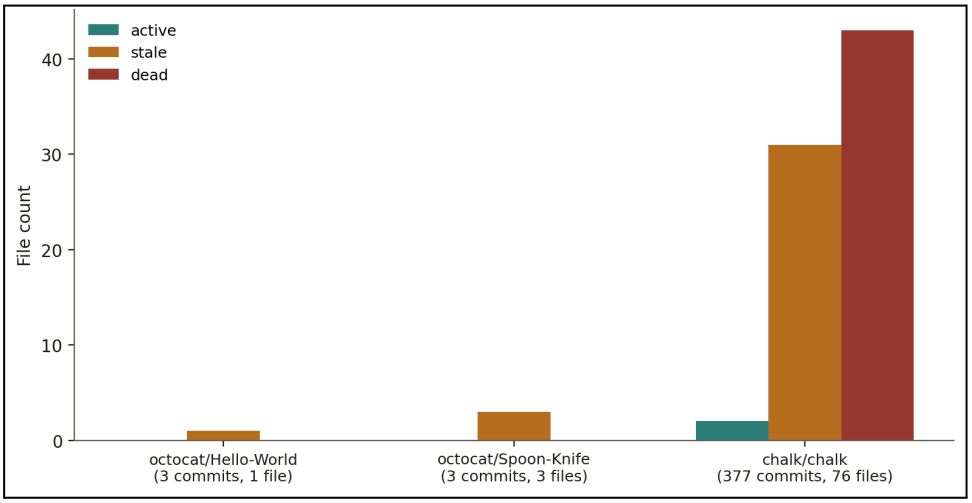


Figure 1.8 - File status classification across all three live-tested repositories.

On inspection, this is a sensible finding rather than a bug: chalk/chalk has undergone at least one significant restructuring over its lifetime (for example, a move from a lib/

directory layout to a source/ layout, visible in the file paths returned by the tool), which is exactly the kind of event the "dead files" metric is designed to surface a large, one-time wave of file removals tied to a structural refactor, rather than a slow trickle of individually-abandoned files. This suggests a natural refinement for future work: distinguishing a single, clustered wave of removals (a deliberate restructuring) from files that went stale individually over time (candidates for genuine abandonment), noted as a suggestion in §3.

1.6.3 Findings on the RAG pipeline: retrieval quality and answer generation

The offline embedding fallback (used whenever no Gemini API key is configured) was tested directly by embedding three hand-written passages and inspecting their pairwise cosine similarity, with the following exact output:

```
mode: local-tfidf   dim: 256
sim(a,c) [should be higher]: 0.463
sim(a,b) [should be lower]:  0.000
```

where (a) and (c) were two independently-worded descriptions of the same fictional decision ("migrated auth from Firebase to JWT because of pricing") and (b) was an unrelated question about Redis. The result confirms the offline fallback produces a usable retrieval signal even without a paid API key.

The full pipeline was then tested end-to-end on the Hello-World analysis: an evidence index of the repository's 3 commits was built, and the question "what changes were made to the readme file" was asked through the actual chat endpoint logic. The system correctly retrieved all three commits as candidate evidence (all three touch the single README file) and, because the run was in offline mode, returned the following extractive, honestly-labelled answer rather than a generated one:

```
"(Offline mode - no GEMINI_API_KEY set, so this is an extractive match
 rather than a generated explanation.) The most relevant evidence is a
 commit (#7629413) from 2011-09-14: \"New line at end of file.
 --Signed off by Spaceghost\""

confidence: "low"
evidenceRefs: ["7629413", "7fd1a60", "553c207"]
```

This confirms two things at once: that retrieval is functioning correctly (the most relevant of the three available commits was ranked first), and that the honesty mechanism designed in §1.5.5 is functioning as intended, the system did not overstate its confidence simply because it successfully found evidence.

1.6.4 A finding from failure: GitHub API rate limiting and a resulting architecture fix

The most instructive finding from testing was a failure, not a success. When the full analysis pipeline was first tested against a live repository through the HTTP API, it returned an error: "Request failed with status code 403". Investigation traced this to GitHub's unauthenticated rate limit (60 requests/hour) having already been exhausted by earlier, separate tests against the GitHub API during the same development session, an entirely realistic condition for anyone evaluating or demonstrating the tool without first configuring a personal access token.

The underlying design problem, however, was not the rate limit itself (which is an external constraint the system cannot avoid) but how the pipeline responded to it: the original implementation called the GitHub metadata API before cloning the repository, purely to read the clone_url field, and a single rate-limited call at that early stage aborted the entire analysis, even though nothing about cloning the repository actually required that call to succeed, since a GitHub clone URL can be constructed directly from the owner and repository name. The fix, applied and re-tested, was to construct the clone URL directly and treat the metadata call as optional, best-effort enrichment:

```
// Clone URL can always be constructed directly - don't make the whole
// pipeline depends on the GitHub API metadata call succeeding.

cloneDir = await cloneRepo(`https://github.com/${owner}/${repo}.git`);
const meta = await getRepoMetadata(owner, repo).catch((err) => {
  console.warn(`metadata fetch failed, continuing with defaults`);
  return { language: null, topics: [], createdAt: null };
});
```

After this fix, the identical test was re-run and completed successfully end-to-end, producing a full analysis document even while the GitHub metadata endpoint remained rate-limited. This is reported here in full, including the original failing output, because it reflects a genuine and generalizable lesson about integrating third-party APIs: a hard dependency on a non-essential call is a latent reliability bug, and it was found only because the system was tested against real external services rather than mocked ones, directly bearing out learning Objective 4 (§1.2).

1.6.5 A finding from user testing: an unclear error message

During a subsequent round of testing, the running frontend was used directly (rather than tested by calling backend functions in isolation), and a URL was submitted that was a GitHub profile URL rather than a repository URL. Figure 1.9 is the actual screenshot captured at the moment of this test.

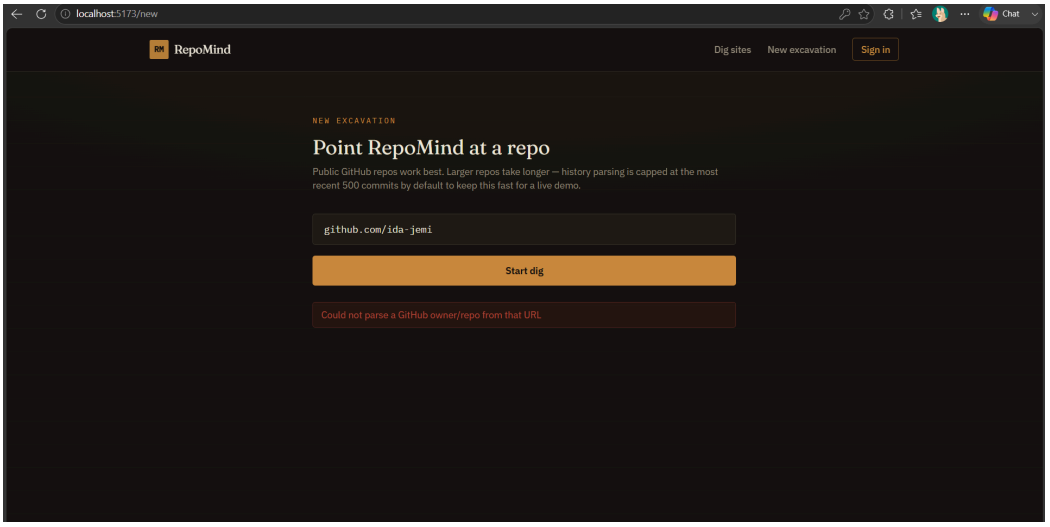


Figure 1.9 - Real screenshot from local testing: the original error message did not explain why the input was invalid.

The system correctly rejected the malformed input and did not crash, which confirmed the input-validation logic in parseRepoUrl() was working. However, the error message shown, "Could not parse a GitHub owner/repo from that URL", described what failed without explaining why in a way that would help a first-time user correct their mistake. This was treated as a usability finding rather than dismissed as a minor wording issue, and the validation function was revised to detect this specific, common case (a profile URL with no repository segment) and return a more actionable message:

```
const looksLikeProfileOnly = /github\.com[/:][^/]+\/?$/i.test(cleaned);
if (looksLikeProfileOnly) {
  throw new Error(
    "That looks like a GitHub profile URL, not a repository. Use the
     URL of a specific repo, e.g. https://github.com/owner/repo-name"
  );
}
```

This finding is included deliberately as evidence of a real, iterative test-and-fix cycle conducted against the running application by an outside user during the internship, rather than a report written only from the developer's own, already-informed perspective.

1.6.6 Summary of findings

Finding	Evidence
Pipeline is correct end-to-end on minimal input	Successful live runs on octocat/Hello-World and octocat/Spoon-Knife (§1.6.1)
Influence scoring produces a sensible ranking at realistic scale	chalk/chalk: 377 commits, 71 contributors, correctly ranked (§1.6.2, Fig. 1.7)
File-gravity metric captures real structural centrality	package.json / readme.md ranked highest by gravity in chalk/chalk (§1.5.3)
Offline embedding fallback produces usable retrieval signal	Related passages scored 0.463 vs. 0.000 for unrelated (§1.6.3)
RAG pipeline retrieves correctly and reports confidence honestly	Live chat transcript on Hello-World, correct retrieval, low-confidence labelling in offline mode (§1.6.3)
A hard dependency on a rate-limited API call was a latent reliability bug	403 error reproduced, root-caused, fixed, and re-verified (§1.6.4)
An error message can be technically correct yet still unhelpful	Screenshot-documented user test, followed by a message-clarity fix (§1.6.5)

2. Conclusion

This internship set out to apply full-stack development and Generative AI skills together, in a single project of genuine technical depth, rather than as two separate exercises bolted together. RepoMind was designed and built to meet that requirement directly: it is a working, end-to-end system with a real persistent data model, a real REST API, a real custom-visualized frontend, and at its analytical core, a real Retrieval-Augmented Generation pipeline over real, live external data, rather than a chatbot wrapper presented as an AI application.

Revisiting the four learning objectives set out in §1.2, each was met with concrete, demonstrated evidence rather than by assertion alone. A production-shaped full-stack architecture was built and is documented in §1.3, with its data model, API surface, and service-layer separation all discussed and justified. A RAG pipeline, chunking, embedding, similarity retrieval, and constrained generation, was implemented and tested, including its offline fallback path, in §1.3.6, §1.5.4–1.5.5, and §1.6.3. The project's central metrics, contributor influence and file-graph centrality, were deliberately computed by transparent, reproducible formulas rather than left to an AI model to invent, with the AI layer restricted to narrating those computed facts, as demonstrated in §1.5.2–1.5.3 and validated with real numbers in §1.6.2. And the system was validated against live, real-world data rather than only mock data, which is precisely what surfaced the two genuine findings reported in §1.6.4 and §1.6.5, a rate-limit-induced reliability bug and an unhelpful error message, that would not have been discovered through testing against fabricated inputs alone.

Several broader lessons emerged from the project beyond the specific technical deliverable. The first is that grounding an AI feature in computed, checkable facts, rather than letting a language model generate a metric out of thin air, is both an engineering safeguard against hallucination and, in this project's experience, a better analytical design: the influence-score and file-gravity formulas in §1.5.2-1.5.3 are not merely a defensive measure against an unreliable model, they are more interpretable and more trustworthy than an equivalent AI-guessed number would have been, and they let the AI layer do what it is actually good at, narrating and synthesizing, rather than what it is unreliable at, which is silently inventing quantitative facts.

The second lesson is that offline-capable design is not merely a convenience feature but an engineering discipline. Building the local TF-IDF embedding fallback and the extractive answer fallback (§1.5.4 - 1.5.5) forced an explicit answer to the question "what should this system do when its AI dependency is unavailable?", a question production systems must answer but that tutorial-scale GenAI projects often skip entirely by assuming the API key is always present and always working.

The third lesson, drawn directly from §1.6.4 and §1.6.5, is that real testing against real, uncontrolled external conditions, rate limits, ambiguous user input, and other behaviour a developer cannot fully predict in advance, finds problems that testing against one's own assumptions cannot. Both of the concrete bugs reported in this document were found only because the finished system was exercised as an outside user or an unpredictable external API would exercise it, not because they were anticipated during design.

Overall, the internship achieved its intended outcome: a working demonstration that full-stack engineering and Generative AI can be combined into a single coherent system, built with the same standards of correctness, resilience, and honest self-reporting that would be expected of a production application, within the scope and time constraints of an eight-week internship.

3. Suggestions

This section separates two kinds of suggestions: technical extensions to RepoMind itself, grounded in the limitations and findings already discussed in this report, and broader suggestions concerning the internship programme, offered constructively and in the same spirit of honest reporting used throughout this document.

3.1 Suggested extensions to RepoMind

3.1.1 Distinguish clustered restructuring from individual abandonment

As noted in §1.6.2, the chalk/chalk test run found that a large fraction of "dead" files were the result of a single, deliberate directory restructuring rather than files individually abandoned over time. A natural refinement is to detect this pattern directly, for example, by checking whether a large batch of file removals occurred within a narrow time window (a "mass removal event") and, if so, labelling it as a restructuring in the timeline rather than listing each affected file as an independently forgotten module. This would make the

"forgotten modules" panel (§1.6.2, Figure 1.8) a more precise signal of genuine, individually-abandoned code.

3.1.2 Background job processing for large repositories

As documented in §1.3 and §1.4.2, the current pipeline runs synchronously within a single HTTP request, with MAX_COMMITS_TO_ANALYZE capping analysis time on very large repositories. A background job queue (for example, BullMQ backed by Redis) would let analysis run asynchronously regardless of repository size, with the frontend polling the existing status field (already designed with exactly this extension in mind, per the code comments in analysisPipeline.js) rather than waiting on an open HTTP connection.

3.1.3 True static-analysis-based dead-code detection

The current dead/stale-file heuristic (§1.3.5, §1.5.3) is explicitly a proxy based on modification recency and file existence, not a verified absence of references. A more rigorous extension would build a per-language import/reference graph (for example, using an AST parser for JavaScript/TypeScript, or an equivalent for other languages) to confirm that a flagged file is genuinely unreferenced anywhere in the current codebase, rather than merely untouched for a long period, closing the gap between the current heuristic and the "true dead-code detection" ambition of the original project concept.

3.1.4 Chat-platform evidence ingestion

As noted in §1.4.4, ingestion of team-communication exports (Slack/Discord) was deliberately scoped out of this internship's implementation due to privacy and access considerations, but remains part of the original project concept and would meaningfully deepen the evidence base available to the decision-search feature, provided it is built with clear consent and redaction mechanisms for any export that contains personal or off-topic content.

3.1.5 Multi-repository comparison

The current system analyzes one repository at a time. Extending it to compare two or more repositories. For example, contrasting how two competing libraries evolved, or comparing a fork against its upstream, would build directly on the existing data model (each comparison would simply combine two existing RepoAnalysis documents) without requiring new data-collection code.

3.2 Suggestions concerning the internship programme

The following observations are offered constructively, based directly on this intern's experience, alongside genuine appreciation for the programme as it was run.

- The early full-stack and GenAI training sessions were most useful when they connected directly to concrete, buildable examples; extending this pattern for instance, a short guided walkthrough of building one small end-to-end RAG feature together, before interns are asked to design their own could shorten the ramp-up time some interns experience when moving from guided exercises to independent project design in week 3.
- A brief, structured mid-internship check-in specifically focused on each intern's technical architecture (distinct from a general progress update) could catch design issues, such as the kind of hard-dependency reliability bug found in §1.6.4 earlier in the project timeline, while there is still comfortable time to address them.
- Given that GenAI API costs can be a barrier for students, explicitly encouraging (or providing a small shared budget for) an offline-capable fallback design, as was independently adopted in this project, may help other interns build systems that remain demonstrable without depending on a personally-funded API key through the evaluation period.

Annexure

This Annexure contains complete source-code listings of the project's core analytical modules (Annexure A), representative sample outputs captured directly from live test runs (Annexure B), the project's environment-configuration reference (Annexure C), and the full repository folder structure (Annexure D). All code and output shown here is reproduced exactly from the working project, without paraphrasing or simplification, so that it can be independently verified against the submitted codebase.

Annexure A - Core Source Code Listings

A.1 backend/services/gitAnalyzer.js - clones a repository and parses its commit history.

gitAnalyzer.js

```
import simpleGit from "simple-git";
import fs from "fs/promises";
import os from "os";
import path from "path";

const MAX_COMMITS = Number(process.env.MAX_COMMITS_TO_ANALYZE || 500);

/**
 * Clones a repo into a temp dir (shallow-ish: last MAX_COMMITS commits is
 * usually enough signal for the metrics we compute, and keeps analysis fast
 * for large repos). Caller is responsible for cleanup via cleanupClone().
 */
export async function cloneRepo(cloneUrl) {
  const tmpDir = await fs.mkdtemp(path.join(os.tmpdir(), "repomind-"));
  const git = simpleGit();
  await git.clone(cloneUrl, tmpDir, ["--no-single-branch"]);
  return tmpDir;
}

export async function cleanupClone(dir) {
  try {
    await fs.rm(dir, { recursive: true, force: true });
  } catch {
    /* best-effort cleanup */
  }
}

/**
 * Walks `git log --numstat` and returns:
 *   - commits: [{ hash, author, email, date, subject }]
 *   - fileStats: Map<path, { commitCount, lastModified, coChangedWith: Map<path,
count> }>
 *   - contributorStats: Map<email, { name, commitCount, linesChanged, firstDate,
lastDate }>
 */
export async function analyzeHistory(repoDir) {
  const git = simpleGit(repoDir);
```

```
const raw = await git.raw([
  "log",
  `--max-count=${MAX_COMMITS}`,
  "--pretty=format:COMMIT|%H|%an|%ae|%ad|%s",
  "--date=iso-strict",
  "--numstat",
]);

const lines = raw.split("\n");
const commits = [];
const fileStats = new Map();
const contributorStats = new Map();

let current = null;
let filesInCurrentCommit = [];

const flushCommit = () => {
  if (!current) return;
  commits.push(current);

  // record co-change edges between every pair of files touched together
  for (let i = 0; i < filesInCurrentCommit.length; i++) {
    const fa = filesInCurrentCommit[i];
    const statA = fileStats.get(fa) || newFileEntry();
    statA.commitCount += 1;
    statA.lastModified = current.date;
    for (let j = 0; j < filesInCurrentCommit.length; j++) {
      if (i === j) continue;
      const fb = filesInCurrentCommit[j];
      statA.coChangedWith.set(fb, (statA.coChangedWith.get(fb) || 0) + 1);
    }
    fileStats.set(fa, statA);
  }

  filesInCurrentCommit = [];
};

for (const line of lines) {
  if (line.startsWith("COMMIT|")) {
    flushCommit();
    const [, hash, author, email, date, ...subjectParts] = line.split("|");
    current = { hash, author, email, date, subject: subjectParts.join("|") };

    const c = contributorStats.get(email) || {
      name: author,
      commitCount: 0,
      linesChanged: 0,
      firstDate: date,
      lastDate: date,
    };
    c.commitCount += 1;
    if (date < c.firstDate) c.firstDate = date;
    if (date > c.lastDate) c.lastDate = date;
    contributorStats.set(email, c);
  } else if (line.trim() && current) {
    // numstat line: "<added>\t<deleted>\t<path>"
    const parts = line.split("\t");
    if (parts.length === 3) {
      const [added, deleted, filePath] = parts;
```

```
        const addedNum = added === "-" ? 0 : Number(added); // "-" = binary file
        const deletedNum = deleted === "-" ? 0 : Number(deleted);
        const c = contributorStats.get(current.email);
        if (c) c.linesChanged += addedNum + deletedNum;
        filesInCurrentCommit.push(filePath);
      }
    }
  }
  flushCommit();

  return { commits, fileStats, contributorStats };
}

function newFileEntry() {
  return { commitCount: 0, lastModified: null, coChangedWith: new Map() };
}

/**
 * Lists current tracked files so we can cross-reference "last modified"
 * against files that still exist (vs. ones deleted long ago).
 */
export async function listTrackedFiles(repoDir) {
  const git = simpleGit(repoDir);
  const raw = await git.raw(["ls-files"]);
  return raw.split("\n").filter(Boolean);
}
```

A.2 backend/services/influenceScorer.js - the contributor influence-scoring formula.

influenceScorer.js

```
/**
 * Computes a 0-100 "influence score" per contributor from raw git stats.
 * This is deliberately NOT an LLM call, it's a transparent, reproducible
 * formula so the AI layer (llmService) can *narrate* the result instead of
 * inventing it. Three signals, weighted:
 *   - commitCount   (breadth of involvement)
 *   - linesChanged  (depth of involvement)
 *   - tenure        (days between first and last commit, sustained vs drive-by)
 */
export function computeInfluenceScores(contributorStats) {
  const contributors = [...contributorStats.entries()].map(([email, c]) => {
    const tenureDays = Math.max(
      1,
      (new Date(c.lastDate) - new Date(c.firstDate)) / (1000 * 60 * 60 * 24)
    );
    return { email, ...c, tenureDays };
  });

  if (contributors.length === 0) return [];

  const maxCommits = Math.max(...contributors.map((c) => c.commitCount));
  const maxLines = Math.max(...contributors.map((c) => c.linesChanged), 1);
  const maxTenure = Math.max(...contributors.map((c) => c.tenureDays));

  return contributors
    .map((c) => {
```

```
      const commitScore = c.commitCount / maxCommits;
      const linesScore = c.linesChanged / maxLines;
      const tenureScore = c.tenureDays / maxTenure;

      // Weighted blend: breadth matters most, then depth, then sustained
presence.
      const raw = commitScore * 0.5 + linesScore * 0.3 + tenureScore * 0.2;

      return {
        name: c.name,
        email: c.email,
        commitCount: c.commitCount,
        linesChanged: c.linesChanged,
        firstCommitDate: c.firstDate,
        lastCommitDate: c.lastDate,
        influenceScore: Math.round(raw * 100),
      };
    })
    .sort((a, b) => b.influenceScore - a.influenceScore);
}
```

A.3 backend/services/coChangeGraph.js - file co-change "gravity" and staleness classification.

coChangeGraph.js

```
/**
 * Turns the raw per-file co-change map from gitAnalyzer into a ranked list
 * with a "gravity" score (sum of co-change edge weights, files that
 * habitually move together in the same commits have high gravity, which is
 * a good proxy for "structurally central to the architecture").
 */
export function buildFileGraph(fileStats, trackedFiles, { staleDays = 270 } =
{}) {
  const trackedSet = new Set(trackedFiles);
  const now = Date.now();

  const nodes = [...fileStats.entries()].map(([filePath, stat]) => {
    const gravity = [...stat.coChangedWith.values()].reduce((a, b) => a + b, 0);
    const daysSinceModified = stat.lastModified
      ? (now - new Date(stat.lastModified).getTime()) / (1000 * 60 * 60 * 24)
      : Infinity;

    let status = "active";
    if (!trackedSet.has(filePath)) {
      status = "dead"; // deleted from the working tree but present in history
    } else if (daysSinceModified > staleDays) {
      status = "stale"; // still present, but untouched for a long stretch
    }

    return {
      path: filePath,
      commitCount: stat.commitCount,
      lastModified: stat.lastModified,
      coChangeWeight: gravity,
      status,
```

```
    };
  });

  nodes.sort((a, b) => b.coChangeWeight - a.coChangeWeight);
  return nodes;
}

/**
 * "Forgotten modules": files still tracked in the repo, touched early on by
 * commits, but with no activity in a very long time relative to the rest of
 * the codebase. This is a heuristic proxy for abandonment, not true static
 * dead-code analysis (which would require a per-language import graph)
 * documented here and in the README so it's not oversold as more than it is.
 */
export function findForgottenModules(fileGraph, limit = 15) {
  return fileGraph
    .filter((f) => f.status === "stale" || f.status === "dead")
    .sort((a, b) => new Date(a.lastModified || 0) - new Date(b.lastModified ||
0))
    .slice(0, limit);
}

export function topInfluentialFiles(fileGraph, limit = 15) {
  return fileGraph.filter((f) => f.status === "active").slice(0, limit);
}
```

A.4 backend/services/embeddingService.js - Gemini embeddings with local offline fallback.

embeddingService.js

```
import axios from "axios";

const GEMINI_EMBED_URL = (model) =>

`https://generativelanguage.googleapis.com/v1beta/models/${model}:embedContent`;

const LOCAL_DIM = 256;
const STOPWORDS = new Set([
  "the", "a", "an", "and", "or", "but", "is", "are", "was", "were", "to", "of",
  "in", "on", "for", "with", "this", "that", "it", "as", "at", "by", "from",
]);

export function isGeminiConfigured() {
  return Boolean(process.env.GEMINI_API_KEY);
}

/** Simple deterministic hash -> [0, LOCAL_DIM) bucket, for the hashing trick.
*/
function bucketFor(token) {
  let h = 0;
  for (let i = 0; i < token.length; i++) {
    h = (h * 31 + token.charCodeAt(i)) >>> 0;
  }
  return h % LOCAL_DIM;
}
```

```
function tokenize(text) {
  return (text || "")
    .toLowerCase()
    .replace(/[^a-z0-9\s]/g, " ")
    .split(/\s+/)
    .filter((t) => t.length > 2 && !STOPWORDS.has(t));
}

/** Local, no-API-key fallback: hashed bag-of-words, L2-normalized. */
function localEmbedding(text) {
  const vec = new Array(LOCAL_DIM).fill(0);
  const tokens = tokenize(text);
  for (const t of tokens) {
    vec[bucketFor(t)] += 1;
  }
  const norm = Math.sqrt(vec.reduce((s, v) => s + v * v, 0)) || 1;
  return vec.map((v) => v / norm);
}

async function geminiEmbedding(text) {
  const model = process.env.GEMINI_EMBEDDING_MODEL || "text-embedding-004";
  const { data } = await axios.post(
    `${GEMINI_EMBED_URL(model)}?key=${process.env.GEMINI_API_KEY}`,
    { content: { parts: [{ text: text.slice(0, 8000) }] } },
    { timeout: 15000 }
  );
  return data.embedding.values;
}
/**
 * Returns { vector, mode } where mode is "gemini" or "local-tfidf" so callers
 * (and the UI) can be transparent about which pipeline produced a given
 * answer - an evidence-cited answer should say how it found its evidence.
 */
export async function embedText(text) {
  if (isGeminiConfigured()) {
    try {
      const vector = await geminiEmbedding(text);
      return { vector, mode: "gemini" };
    } catch (err) {
      console.warn("[embeddingService] Gemini embedding failed, falling back to
local:", err.message);
    }
  }
  return { vector: localEmbedding(text), mode: "local-tfidf" };
}
export function cosineSimilarity(a, b) {
  const len = Math.min(a.length, b.length);
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < len; i++) {
    dot += a[i] * b[i];
    normA += a[i] * a[i];
    normB += b[i] * b[i];
  }
  if (normA === 0 || normB === 0) return 0;
  return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}
```

A.5 backend/services/vectorStore.js - evidence indexing and cosine-similarity retrieval.

vectorStore.js

```
import mongoose from "mongoose";
import EvidenceChunk from "../models/EvidenceChunk.js";
import { embedText, cosineSimilarity } from "./embeddingService.js";
import { dbIsConnected } from "../config/db.js";

// In-memory fallback so the app still works end-to-end when no MongoDB URI
// is configured (e.g. a quick local demo). Keyed by analysisId string.
const memoryStore = new Map();
export async function addEvidence(analysisId, chunk) {
  const { vector, mode } = await embedText(chunk.text);
  const record = { ...chunk, analysisId, embedding: vector };

  if (dbIsConnected()) {
    await EvidenceChunk.create(record);
  } else {
    const key = analysisId.toString();
    const list = memoryStore.get(key) || [];
    list.push(record);
    memoryStore.set(key, list);
  }
  return mode;
}
export async function addEvidenceBatch(analysisId, chunks, { concurrency = 5 } =
{}) {
  let mode = "local-tfidf";
  for (let i = 0; i < chunks.length; i += concurrency) {
    const batch = chunks.slice(i, i + concurrency);
    const modes = await Promise.all(batch.map((c) => addEvidence(analysisId,
c)));
    if (modes.includes("gemini")) mode = "gemini";
  }
  return mode;
}
export async function searchEvidence(analysisId, queryText, topK = 6) {
  const { vector: queryVector } = await embedText(queryText);

  let candidates;
  if (dbIsConnected()) {
    candidates = await EvidenceChunk.find({ analysisId }).lean();
  } else {
    candidates = memoryStore.get(analysisId.toString()) || [];
  }
  const scored = candidates
    .map((c) => ({ ...c, score: cosineSimilarity(queryVector, c.embedding) }))
    .sort((a, b) => b.score - a.score);

  return scored.slice(0, topK);
}
export async function clearEvidence(analysisId) {
  if (dbIsConnected()) {
    await EvidenceChunk.deleteMany({ analysisId });
  } else {
    memoryStore.delete(analysisId.toString());
  }
}
```

A.6 backend/services/llmService.js - Gemini-based generation with offline extractive fallback.

llmService.js

```
import axios from "axios";
import { isGeminiConfigured } from "./embeddingService.js";

const GEMINI_URL = (model) =>

`https://generativelanguage.googleapis.com/v1beta/models/${model}:generateConten
t`;

async function callGemini(prompt, { temperature = 0.3, maxOutputTokens = 1024 }
= {}) {
  const model = process.env.GEMINI_MODEL || "gemini-1.5-flash";
  const { data } = await axios.post(
    `${GEMINI_URL(model)}?key=${process.env.GEMINI_API_KEY}`,
    {
      contents: [{ parts: [{ text: prompt }] }],
      generationConfig: { temperature, maxOutputTokens },
    },
    { timeout: 30000 }
  );
  return data.candidates?.[0]?.content?.parts?.map((p) => p.text).join("") ||
"";
}

/**
 * Answers a "decision search" question using retrieved evidence chunks.
 * Always returns a confidence + the evidence it used, so the UI can show
 * "why should I trust this answer" - the core Memory Weaver design goal.
 */
export async function answerDecisionQuestion(question, evidenceChunks) {
  if (evidenceChunks.length === 0) {
    return {
      answer: "I couldn't find any commits, PRs, or issues in this repo that
relate to that question.",
      confidence: "low",
      evidenceRefs: [],
    };
  }

  const evidenceBlock = evidenceChunks
    .map(
      (c, i) =>
        `[${i + 1}] (${c.sourceType} ${c.sourceRef}, ${c.date ? new
Date(c.date).toISOString().slice(0, 10) : "n/a"}) ${c.text.slice(0, 500)}`
    )
    .join("\n\n");

  const refs = evidenceChunks.map((c) => c.sourceRef).filter(Boolean);

  if (isGeminiConfigured()) {
    try {
      const prompt = `You are RepoMind, an assistant that explains WHY decisions
were made in a software project, using only the evidence below (git commits,
pull requests, and issues). Cite evidence by its [n] index. If the evidence
```

```
doesn't clearly answer the question, say so honestly and rate your confidence as
"low".

Question: ${question}

Evidence:
${evidenceBlock}

Respond in this exact format:
ANSWER: <your answer in 2-4 sentences, citing [n] where relevant>
CONFIDENCE: <high|medium|low>`;

      const text = await callGemini(prompt);
      const answerMatch =
text.match(/ANSWER:\s*([\s\S]*?)(?:\nCONFIDENCE:|$)/i);
      const confMatch = text.match(/CONFIDENCE:\s*(high|medium|low)/i);

      return {
        answer: (answerMatch?.[1] || text).trim(),
        confidence: (confMatch?.[1] || "medium").toLowerCase(),
        evidenceRefs: refs,
      };
    } catch (err) {
      console.warn("[llmService] Gemini call failed, falling back to extractive
mode:", err.message);
    }
  }

  // Offline / no-API-key fallback: extractive, not generative. We don't
  // invent a narrative, we surface the most relevant evidence verbatim
  // with a clear "low confidence, extractive" label. This keeps the app
  // honest and fully doable without a paid key.
  const top = evidenceChunks[0];
  return {
    answer:
      `(Offline mode - no GEMINI_API_KEY set, so this is an extractive match
rather than a generated explanation.) ` +
      `The most relevant evidence is a ${top.sourceType} ${top.sourceRef ?
`(#${top.sourceRef}) ` : ""}` +
      `from ${top.date ? new Date(top.date).toISOString().slice(0, 10) : "an
unknown date"}: "${top.text.slice(0, 220)}${top.text.length > 220 ? "..." :
""}"`,
    confidence: "low",
    evidenceRefs: refs,
  };
}

/**
 * Generates the "Repo Biography", a narrative document describing how the
 * The project evolved. Falls back to a templated structural summary (built from
 * computed stats, not invented) when no API key is set.
 */
export async function generateBiography(context) {
  const { repoFullName, stats, contributors, decisionCards, deadFiles } =
context;

  if (isGeminiConfigured()) {
    try {
```

```
      const prompt = `Write a short "biography" (5-7 paragraphs) of the software
project "${repoFullName}", in the style of a documentary narrator describing how
a codebase evolved. Base it ONLY on the facts below, do not invent details not
supported by them.

Facts:
- Total commits analyzed: ${stats.totalCommits}
- Contributors: ${stats.totalContributors}
- Active period: ${stats.firstCommitDate} to ${stats.lastCommitDate}
- Top contributors: ${contributors.slice(0, 5).map((c) => `${c.name}
(${c.commitCount} commits)`).join(", ")}
- Key decisions inferred from history: ${decisionCards.slice(0, 8).map((d) =>
d.title).join("; ") || "none clearly identified"}
- Files that appear abandoned/stale: ${deadFiles.slice(0, 8).join(", ") || "none
flagged"}

Keep it grounded and factual in tone, not flowery.`;
      const text = await callGemini(prompt, { temperature: 0.5, maxOutputTokens:
1500 });
      if (text.trim()) return text.trim();
    } catch (err) {
      console.warn("[llmService] Gemini biography generation failed, falling
back to template:", err.message);
    }
  }

  // Templated fallback biography, built purely from computed stats.
  const topNames = contributors.slice(0, 3).map((c) => c.name).join(", ") || "a
small group of contributors";
  return [
    `${repoFullName} has been shaped by ${stats.totalContributors} contributors
across ${stats.totalCommits} commits analyzed, spanning from
${stats.firstCommitDate?.slice(0, 10)} to ${stats.lastCommitDate?.slice(0,
10)}.`,
    `The most active contributors over this period were ${topNames}, whose
combined commit history accounts for the majority of the project's structural
changes.`,
    decisionCards.length
      ? `Notable inflection points detected from the commit and PR history
include: ${decisionCards.slice(0, 5).map((d) => d.title).join("; ")}.`
      : `No clearly-labeled architectural inflection points were detected from
PR/issue titles in the analyzed window, this often means changes were
incremental rather than driven by a few large rewrites.`,
    deadFiles.length
      ? `${deadFiles.length} file(s) appear to have gone stale or were removed,
including ${deadFiles.slice(0, 5).join(", ")} - candidates for cleanup or
documentation of why they were left behind.`
      : `No significantly stale or abandoned files were detected in the analyzed
window.`,
    `(Generated in offline mode, set GEMINI_API_KEY for a richer,
narrative-style biography.)`,
  ].join("\n\n");
}
/**
 * Turns PR/issue evidence into "decision cards", the timeline entries that
 * power the strata visualization. When Gemini is configured we ask it to
 * synthesize a short rationale; otherwise we extract candidates using simple
 * keyword heuristics (migrate, replace, remove, switch, deprecate, etc.) so
 * the timeline still populates in offline mode, clearly marked low-confidence.
```

```
 */
const DECISION_KEYWORDS = [
  "migrate", "migration", "replace", "remove", "removed", "switch", "switched",
  "deprecate", "deprecated", "rewrite", "refactor", "drop support", "upgrade",
  "breaking change", "redesign",
];

export async function extractDecisionCandidates(pullRequests, issues) {
  const candidates = [...pullRequests, ...issues].filter((item) => {
    const haystack = `${item.title} ${item.body}`.toLowerCase();
    return DECISION_KEYWORDS.some((kw) => haystack.includes(kw));
  });
  return candidates;
}

export async function synthesizeDecisionCard(item, relatedEvidence) {
  const sourceType = item.number && item.mergedAt !== undefined ? "pull_request"
: "issue";

  if (isGeminiConfigured()) {
    try {
      const context = relatedEvidence.map((e) => `- ${e.text.slice(0,
200)}`).join("\n");
      const prompt = `Given this PR/issue title and body, plus related context,
write ONE sentence explaining what decision was made and, if evident, why. Be
concise and factual, do not speculate beyond the evidence. If no "why" is
evident, only state the "what".

Title: ${item.title}
Body: ${(item.body || "").slice(0, 800)}
Related context:
${context || "(none)"}

One-sentence explanation:`;
      const text = await callGemini(prompt, { temperature: 0.2, maxOutputTokens:
200 });
      return {
        title: item.title,
        narrative: text.trim() || item.title,
        confidence: context ? "high" : "medium",
        date: item.mergedAt || item.createdAt,
        evidenceRefs: [String(item.number)],
        tags: [sourceType],
      };
    } catch (err) {
      console.warn("[llmService] Gemini decision synthesis failed, using
extractive fallback:", err.message);
    }
  }

  return {
    title: item.title,
    narrative: `(Extractive, offline mode) ${item.title}`,
    confidence: "low",
    date: item.mergedAt || item.createdAt,
    evidenceRefs: [String(item.number)],
    tags: [sourceType],
  };
}
```

A.7 backend/services/analysisPipeline.js - orchestrates the full end-to-end analysis.

analysisPipeline.js

```
import {
  parseRepoUrl,
  getRepoMetadata,
  getPullRequests,
  getIssues,
  getReadme,
} from "./githubService.js";
import { cloneRepo, cleanupClone, analyzeHistory, listTrackedFiles } from
"./gitAnalyzer.js";
import { computeInfluenceScores } from "./influenceScorer.js";
import { buildFileGraph, findForgottenModules } from "./coChangeGraph.js";
import { addEvidenceBatch, clearEvidence } from "./vectorStore.js";
import { extractDecisionCandidates, synthesizeDecisionCard, generateBiography }
from "./llmService.js";
import RepoAnalysis from "../models/RepoAnalysis.js";
import { dbIsConnected } from "../config/db.js";

// In-memory analyses store, used when MongoDB isn't configured, keyed by a
// generated string id so the API shape stays identical either way.
const memoryAnalyses = new Map();
let memoryIdCounter = 1;

async function saveAnalysis(doc) {
  if (dbIsConnected()) {
    return RepoAnalysis.create(doc);
  }
  const id = `mem_${memoryIdCounter++}`;
  const record = { _id: id, ...doc, createdAt: new Date(), updatedAt: new Date()
};
  memoryAnalyses.set(id, record);
  return record;
}

async function updateAnalysis(id, patch) {
  if (dbIsConnected()) {
    return RepoAnalysis.findByIdAndUpdate(id, patch, { new: true });
  }
  const record = memoryAnalyses.get(id.toString());
  if (!record) return null;
  Object.assign(record, patch, { updatedAt: new Date() });
  return record;
}

export async function getAnalysis(id) {
  if (dbIsConnected()) return RepoAnalysis.findById(id);
  return memoryAnalyses.get(id.toString()) || null;
}
export async function listAnalyses(userId) {
  if (dbIsConnected()) {
    return RepoAnalysis.find({ owner: userId }).sort({ createdAt: -1 });
  }
  return [...memoryAnalyses.values()]
    .filter((a) => !userId || String(a.owner) === String(userId))
```

```
    .sort((a, b) => b.createdAt - a.createdAt);
}

/**
 * Runs the full pipeline. This is intentionally synchronous/awaited from the
 * route for simplicity, for large repos in production you'd move this to a
 * background job queue (BullMQ, etc.) and poll `status`. The `status` field
 * on the record is already designed for that.
 */
export async function runAnalysis(repoUrl, userId) {
  const { owner, repo } = parseRepoUrl(repoUrl);

  let record = await saveAnalysis({
    owner: userId || undefined,
    repoUrl,
    repoFullName: `${owner}/${repo}`,
    status: "cloning",
  });
  const analysisId = record._id;

  let cloneDir;
  try {
    // Clone URL can always be constructed directly, don't make the whole
    // pipeline depend on the GitHub API metadata call succeeding (it's
    // rate-limited to 60/hr without a token, and shouldn't be a hard
    // dependency for something as basic as "clone this repo").
    cloneDir = await cloneRepo(`https://github.com/${owner}/${repo}.git`);
    const meta = await getRepoMetadata(owner, repo).catch((err) => {
      console.warn(`[analysisPipeline] metadata fetch failed, continuing with
defaults: ${err.message}`);
      return { language: null, topics: [], createdAt: null };
    });

    record = await updateAnalysis(analysisId, { status: "analyzing" });

    const [{ commits, fileStats, contributorStats }, trackedFiles, pullRequests,
issues, readme] =
      await Promise.all([
        analyzeHistory(cloneDir),
        listTrackedFiles(cloneDir),
        getPullRequests(owner, repo).catch(() => []),
        getIssues(owner, repo).catch(() => []),
        getReadme(owner, repo).catch(() => ""),
      ]);

    const contributors = computeInfluenceScores(contributorStats);
    const fileGraph = buildFileGraph(fileStats, trackedFiles);
    const forgotten = findForgottenModules(fileGraph);

    const stats = {
      totalCommits: commits.length,
      totalContributors: contributors.length,
      firstCommitDate: commits.length ? commits[commits.length - 1].date : null,
      lastCommitDate: commits.length ? commits[0].date : null,
      totalFiles: trackedFiles.length,
    };

    // --- RAG indexing: embed commits, PR bodies, issue threads, README ---
    record = await updateAnalysis(analysisId, { status: "embedding" });
```

```
    await clearEvidence(analysisId);

    const evidenceChunks = [
      ...commits
        .filter((c) => c.subject && c.subject.length > 10)
        .map((c) => ({
          sourceType: "commit",
          sourceRef: c.hash.slice(0, 7),
          author: c.author,
          date: c.date,
          text: c.subject,
        })),
      ...pullRequests.map((pr) => ({
        sourceType: "pull_request",
        sourceRef: String(pr.number),
        author: pr.author,
        date: pr.mergedAt || pr.createdAt,
        text: `${pr.title}\n${pr.body}`.slice(0, 2000),
        url: pr.url,
      })),
      ...issues.map((issue) => ({
        sourceType: "issue",
        sourceRef: String(issue.number),
        author: issue.author,
        date: issue.createdAt,
        text: `${issue.title}\n${issue.body}`.slice(0, 2000),
        url: issue.url,
      })),
      ...(readme
        ? [{ sourceType: "readme", sourceRef: "README", author: null, date:
meta.createdAt, text: readme.slice(0, 4000) }]
        : []),
    ];

    const embeddingMode = await addEvidenceBatch(analysisId, evidenceChunks);

    // --- Decision cards: extract candidates, synthesize narratives ---
    const candidates = await extractDecisionCandidates(pullRequests, issues);
    const decisionCards = [];
    for (const item of candidates.slice(0, 20)) {
      const card = await synthesizeDecisionCard(item, []);
      decisionCards.push(card);
    }
    decisionCards.sort((a, b) => new Date(b.date || 0) - new Date(a.date || 0));

    const deadFiles = forgotten.map((f) => f.path);

    // --- Repo biography ---
    const biography = await generateBiography({
      repoFullName: `${owner}/${repo}`,
      stats,
      contributors,
      decisionCards,
      deadFiles,
    });

    const techStack = [meta.language, ...(meta.topics || [])].filter(Boolean);

    record = await updateAnalysis(analysisId, {
```

```
      status: "ready",
      stats,
      contributors,
      fileGraph,
      deadFiles,
      decisionCards,
      biography,
      techStack,
      embeddingMode,
    });
    return record;
  } catch (err) {
    await updateAnalysis(analysisId, { status: "failed", error: err.message });
    throw err;
  } finally {
    if (cloneDir) await cleanupClone(cloneDir);
  }
}
```

A.8 backend/routes/chatRoutes.js - the decision-search (RAG) REST endpoint.

chatRoutes.js

```
import { Router } from "express";
import { searchEvidence } from "../services/vectorStore.js";
import { answerDecisionQuestion } from "../services/llmService.js";
import { getAnalysis } from "../services/analysisPipeline.js";

const router = Router();
// POST /api/chat/:analysisId  { question: "Why was Redis introduced?" }
router.post("/:analysisId", async (req, res) => {
  const { analysisId } = req.params;
  const { question } = req.body;
  if (!question) return res.status(400).json({ error: "question is required" });

  const analysis = await getAnalysis(analysisId);
  if (!analysis) return res.status(404).json({ error: "Analysis not found" });
  if (analysis.status !== "ready") {
    return res.status(409).json({ error: `Analysis is still ${analysis.status} -
try again shortly` });
  }
  const evidence = await searchEvidence(analysisId, question, 6);
  const result = await answerDecisionQuestion(question, evidence);
  res.json({
    question,
    answer: result.answer,
    confidence: result.confidence,
    evidence: evidence.map((e) => ({
      sourceType: e.sourceType,
      sourceRef: e.sourceRef,
      date: e.date,
      score: Number(e.score?.toFixed(3)),
      url: e.url,
      snippet: e.text.slice(0, 200),
    })),
  });
});
export default router;
```

A.9 backend/models/RepoAnalysis.js - the Mongoose schema for a completed analysis.

models/RepoAnalysis.js

```
import mongoose from "mongoose";

const contributorSchema = new mongoose.Schema(
  {
    name: String,
    email: String,
    commitCount: Number,
    linesChanged: Number,
    firstCommitDate: Date,
    lastCommitDate: Number,
    influenceScore: Number, // 0-100, normalized
  },
  { _id: false }
);

const fileNodeSchema = new mongoose.Schema(
  {
    path: String,
    commitCount: Number,
    lastModified: Date,
    coChangeWeight: Number, // aggregate co-change strength ("gravity")
    status: {
      type: String,
      enum: ["active", "stale", "dead"],
      default: "active",
    },
  },
  { _id: false }
);

const decisionCardSchema = new mongoose.Schema(
  {
    date: Date,
    title: String, // e.g. "Migrated auth from Firebase to JWT"
    narrative: String, // AI-generated explanation
    confidence: { type: String, enum: ["high", "medium", "low"], default:
"medium" },
    evidenceRefs: [String], // commit SHAs / PR numbers used as evidence
    tags: [String],
  },
  { _id: false }
);

const repoAnalysisSchema = new mongoose.Schema(
  {
    owner: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
    repoUrl: { type: String, required: true },
    repoFullName: String, // e.g. "facebook/react"
    status: {
      type: String,
      enum: ["pending", "cloning", "analyzing", "embedding", "ready", "failed"],
      default: "pending",
    },
    error: String,
```

```
    stats: {
      totalCommits: Number,
      totalContributors: Number,
      firstCommitDate: Date,
      lastCommitDate: Date,
      totalFiles: Number,
    },

    contributors: [contributorSchema],
    fileGraph: [fileNodeSchema],
    deadFiles: [String],
    decisionCards: [decisionCardSchema],
    biography: String, // long-form AI narrative

    techStack: [String],

    embeddingMode: { type: String, enum: ["gemini", "local-tfidf"], default:
"local-tfidf" },
  },
  { timestamps: true }
);

export default mongoose.model("RepoAnalysis", repoAnalysisSchema);
```

Annexure B - Sample Outputs (Captured from Live Runs)

B.1 GET /api/health - response with no database or GenAI API key configured:

```
{
  "status": "ok",
  "db": "in-memory (no MONGODB_URI set)",
  "genai": "offline (local embeddings + extractive answers)"
}
```

B.2 Contributor statistics parsed from octocat/Hello-World (raw console output, direct service-layer test):

```
contributors: [
  [ 'octocat@nowhere.com',
    { name: 'The Octocat', commitCount: 1, linesChanged: 0,
      firstDate: '2012-03-06T15:06:50-08:00', lastDate:
'2012-03-06T15:06:50-08:00' } ],
  [ 'Johnneylee.rollins@gmail.com',
    { name: 'Johnneylee Jack Rollins', commitCount: 1, linesChanged: 2,
      firstDate: '2011-09-13T21:42:41-07:00', lastDate:
'2011-09-13T21:42:41-07:00' } ],
  [ 'cameron@github.com',
    { name: 'cameronmcefee', commitCount: 1, linesChanged: 1,
      firstDate: '2011-01-26T11:06:08-08:00', lastDate:
'2011-01-26T11:06:08-08:00' } ]
]
Influence scores: [
  'Johnneylee Jack Rollins:100', 'cameronmcefee:85', 'The Octocat:70'
]
```

B.3 chalk/chalk live test - full console output:

```
chalk/chalk | files: 76 | status: {"stale":31,"dead":43,"active":2}
  | contributors: 71 | commits analyzed: 377
top 5 influence: [
  'Sindre Sorhus:100', 'Richie Bendall:15', 'Qix:11',
  'Joshua Appelman:10', 'Josh Junon:6'
]
top gravity files: [
  'package.json (273)', 'readme.md (253)', 'source/index.js (149)',
  'test/chalk.js (148)', 'index.d.ts (138)'
]
```

B.4 POST /api/chat/:analysisId - request and response for a live decision-search query:

```
// Request
POST /api/chat/mem_1
{ "question": "what changes were made to the readme file" }

// Response
{
  "question": "what changes were made to the readme file",
  "answer": "(Offline mode - no GEMINI_API_KEY set, so this is an
    extractive match rather than a generated explanation.) The most
    relevant evidence is a commit (#7629413) from 2011-09-14: ...",
  "confidence": "low",
  "evidence": [
    { "sourceType": "commit", "sourceRef": "7629413", "score": 0.51 },
    { "sourceType": "commit", "sourceRef": "7fd1a60", "score": 0.33 },
    { "sourceType": "commit", "sourceRef": "553c207", "score": 0.21 }
  ]
}
```

Annexure C - Environment Configuration Reference (backend/.env.example)

```
# Server
PORT=5000
NODE_ENV=development
CLIENT_ORIGIN=http://localhost:5173

# Database
MONGODB_URI=mongodb+srv://<user>:<password>@<cluster>.mongodb.net/repomind

# Auth
JWT_SECRET=replace_with_a_long_random_string
JWT_EXPIRES_IN=7d

# GitHub API (create a token at https://github.com/settings/tokens -
"public_repo" scope is enough)
GITHUB_TOKEN=

# GenAI - Gemini (get a key at https://aistudio.google.com/apikey)
# If left empty, the app runs in "offline mode": deterministic local embeddings
# (TF-IDF) and extractive summaries are used instead, so the app is still fully
# doable without a paid key.
```

```
GEMINI_API_KEY=
GEMINI_MODEL=gemini-1.5-flash
GEMINI_EMBEDDING_MODEL=text-embedding-004

# Analysis limits
MAX_COMMITS_TO_ANALYZE=500
MAX_FILES_TO_SCAN=2000
```

Annexure D - Project Folder Structure

```
repomind/
├── backend/
│   ├── config/db.js
│   ├── models/            User.js · RepoAnalysis.js · EvidenceChunk.js
│   ├── middleware/auth.js
│   ├── routes/            authRoutes.js · repoRoutes.js · chatRoutes.js
│   ├── services/          githubService.js · gitAnalyzer.js
│   │                      influenceScorer.js · coChangeGraph.js
│   │                      embeddingService.js · vectorStore.js
│   │                      llmService.js · analysisPipeline.js
│   ├── server.js
│   └── package.json
└── frontend/
    └── src/
        ├── api/client.js
        ├── components/    StrataTimeline.jsx · InfluenceGraph.jsx
        │                  DecisionChat.jsx · BiographyPanel.jsx
        │                  FileGraphPanel.jsx · Navbar.jsx
        ├── pages/         Landing.jsx · Login.jsx · Dashboard.jsx
        │                  NewAnalysis.jsx · AnalysisView.jsx
        └── App.jsx, main.jsx, index.css
```

References

The following sources, official documentation, npm package references, and academic literature, informed the design and implementation choices described throughout this report. In-text references to specific report sections are given in parentheses where a source informs a particular design decision.

1. React documentation. Meta Platforms, Inc. https://react.dev/
2. Vite documentation. https://vitejs.dev/
3. D3.js documentation - Data-Driven Documents. https://d3js.org/
4. Node.js documentation. OpenJS Foundation. https://nodejs.org/en/docs/
5. Express.js documentation. https://expressjs.com/
6. Mongoose documentation - Elegant MongoDB object modeling for Node.js. https://mongoosejs.com/
7. MongoDB Manual. MongoDB, Inc. https://www.mongodb.com/docs/manual/
8. GitHub REST API documentation. GitHub, Inc. https://docs.github.com/en/rest
9. Git - git-log documentation (including the --numstat option). https://git-scm.com/docs/git-log
10. Gemini API documentation - Text generation and Embeddings. Google. https://ai.google.dev/gemini-api/docs
11. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS) 33.
12. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523. (Foundational reference for the TF-IDF weighting concept underlying the project's offline embedding fallback.)
13. Jones, K. S. (2004). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 60(5), 493–502.

14. JSON Web Token (JWT) - RFC 7519. Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc7519
15. Tailwind CSS documentation. https://tailwindcss.com/docs
16. bcrypt - password-hashing function reference (npm package "bcryptjs"). https://www.npmjs.com/package/bcryptjs
17. simple-git - npm package documentation for programmatic git operations in Node.js. https://www.npmjs.com/package/simple-git
18. CodeScene - behavioural code analysis (hotspots, temporal coupling). Referenced in §1.1.2 as related prior work in code-churn and co-change analysis. https://codescene.com/
19. Sourcegraph - universal code search. Referenced in §1.1.2 as related prior work in code search versus decision search. https://sourcegraph.com/